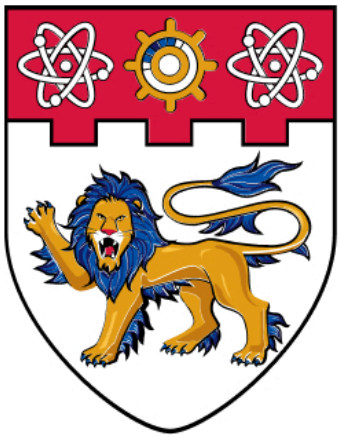


NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

# SC4053 - Blockchain Technology

## Development Project (Option 2)

## AI Audit Report


| Name | Matriculation Number |
|---|---|
| Syed Ridwan Kabir | U2321342C |
| Yojin Oh | U2120115D |

**How to read this report:** Every claim we make cites a file in our repository. Raw logs are in *audit/evidence/*, the figures in *audit/figures/,* our prompt logs in *prompts/*, and the full log of AI mistakes in *audit/ai-issues.md*. We re-ran every test suite, coverage, Slither and the end-to-end test on the final submission on 1 November 2026 (*audit/evidence/2026-11-01-review/*). The results match the session 05 logs (*audit/evidence/2026-10-31-**), so the numbers below are reproducible.

**Github Link:** https://github.com/SyedRidwan/sc4053_group_project_syed_yj.git

## Table of Contents


0. Summary
1. Initial Specification & System Scope
2. Development Methodology & AI Tooling Infrastructure
3. Specification Engineering & Prompt Logs
4. AI-Generated Architecture
5. Problems Discovered & Vulnerability Analysis
6. Validation Process & Testing Strategy
7. Improvements & Refinements
8. Lessons Learned
9. Appendix A — Evidence index
10. Appendix B — Reproducing the evidence
11. Appendix C — Brief and rubric compliance map

## 0. Summary

Horton's Launchpad lets anyone launch an ERC20 token from a website. Every token sells in its own 20-minute, uniform-clearing-price Dutch auction. Bidders commit ETH while the price falls, and the auction clears the moment the committed ETH buys the whole supply. Everyone pays the same clearing price, the bid that crosses the line has its surplus refunded in the same transaction, and unsold tokens are burned.

**Headline results:**

| Check | Result |
|---|---|
| Contract tests (Hardhat, Mocha) | 87 passing, 0 failing |
| Coverage | 100% of lines and statements in every contract. Branches: 98.04% overall (LaunchFactory 100%, DutchAuction 96.97%) |
| Slither 0.11.6 (102 detectors) | 30 results, 0 high, all triaged. The CI gate *--fail-high* passes |
| Mutation testing | 23 of 23 planted bugs caught: 9 of 9 in the auction, 14 of 14 in the factory. Three were caught only after we closed the gaps they exposed |
| Frontend unit tests (Vitest) | 39 passing |
| End-to-end UI test (headless Chromium) | 17 of 17 steps passing across three listings and three wallets |
| Gas | bid at most 152,993; claim at most 161,965; createLaunch about 1.79 million. No function loops over bidders |
| AI mistakes logged | 28 (6 high, 14 medium, 8 low), each in the brief's four-part format |
| Final independent review | 5 residual risks probed (R1 to R5). No new high-severity defect |

**Who produced what?** The brief asks us to audit AI-generated output, so we are precise about provenance. It comes from us and from our prompt logs, never from inference (see AI-2).

| Work | Produced by | Source |
|---|---|---|
| **v1:** spec.md v1, NTUToken.sol, DutchAuction.sol, ReentrancyAttacker.sol and 4 tests (commits 138f1ce, 5f6a2c4, ef71e56) | **Written by us, by hand, before any AI tool** | prompts/2026-10-22-session-01.md |
| **v2 onwards:** spec v2, the rewritten contracts, tests, CLI tasks, frontend and CI (feat/v2-committed-capital, then soln-combined) | **Claude Code**, directed by us | prompts/, sessions 01 to 03 |

| | | |
|---|---|---|
| dutch-auction-token-launch branch (commit 792b50d, author yojin5) | **An AI tool**. We did not record which one | prompts/2026-10-28-session-03.md, prompt 2 |
| **Launchpad:** spec v3, LaunchFactory.sol, the Market, Create, Portfolio and How it works pages, seed and demo tooling | **Claude Code** (multi-agent), directed by us | prompts/2026-10-30-session-04.md |
| Final audit against the brief, fixes for AI-26 to AI-28, first demo storyboard | **Claude Code**, directed by us | prompts/2026-10-31-session-05.md |
| Final re-run on the submitted code, probes R1 to R5, this report's final edition, the demo script and slides | **Claude** (claude.ai), directed and reviewed by us | prompts/2026-11-01-session-06.md |




*Figure 0.1 — How the project developed. v1 is our human baseline. Everything after it is AI-assisted and audited in this report.*

## 1. Initial Specification and System Scope

### 1.1 What we set out to build, before using any AI tool

Before using AI, we wrote a specification, three contracts and four tests by hand. The spec is *commit 138f1ce*, and *ef71e56* is our last hand-written commit. The scope was the brief's Option 2:

| Area | What we intended in v1 |
|---|---|
| Token | An ERC20 with 18 decimals, burnable, with a fixed supply minted to the seller (NTUToken) |
| Auction engine | A separate DutchAuction contract. The price falls linearly from a start price to a reserve over 20 minutes |
| Allocation rule | Each bid **locks in tokens at the price at that moment** (spec.md v1, lines 12 and 95) |
| Bridging bid | A bid that crosses the remaining supply is partly filled, and the excess refunded |
| Settlement | All winners settle at one clearing price, and get the difference refunded when they claim (claimTokensAndRefund) |
| Burn | Unsold tokens are burned after 20 minutes. We implemented this as a transfer to address(0) |
| Security | Reentrancy resistance, shown by a ReentrancyAttacker test (the brief's bonus) |
| Tests | Four lifecycle tests: price decay, the bridging bid, the burn, and reentrancy |

### 1.2 Intended UI features

v1 contained no frontend code. We had not written a separate UI plan, because the brief already lists what the site must do. We planned one page where a user could:

- connect MetaMask;
- watch the live, falling price and the 20-minute countdown;
- place a bid and see any refund;
- claim tokens after the auction settles, and see how many unsold tokens were burned;
- and, as the seller, withdraw the ETH raised.

Multiple listings, a token-launch form, a portfolio page, the auction simulator and the light theme all came later, as deliberate scope decisions (Section 7).

### 1.3 Our baseline did not work

The first AI session ran our hand-written suite: **0 of 4 tests passed** (Figure 1.1). An AI-assisted review then found **13 issues** in v1, five of them critical: each could drain funds or lock them for good. These are *our* mistakes, so they are not part of the AI

inventory in Section 5. They matter here for two reasons. They are why we moved to an AI-assisted rewrite, and issue 7 exposed a flaw **in our own specification**.

audit/evidence/2026-10-22-baseline-test-run.txt · v1 (hand-written, commit ef71e56)

```
$ npx hardhat test


  DutchAuction Full Lifecycle & Security Suite
    1) 1. Enforces exact linear price decay without integer truncation drift
    2) 2. Handles threshold-bridging bids and settles all bidders at uniform clearing price
    3) 3. Permanently burns unsold tokens from ERC20 totalSupply after 20 minutes
    4) 4. Blocks reentrancy attack during ETH refund callback


  0 passing (385ms)
  4 failing

  1) DutchAuction Full Lifecycle & Security Suite
       1. Enforces exact linear price decay without integer truncation drift:

      AssertionError: expected 60000000000000400 to equal 60000000000000000.
      + expected - actual

      -60000000000000400
      +60000000000000000

      at Context.<anonymous> (test\DutchAuction.test.js:36:48)

  2) DutchAuction Full Lifecycle & Security Suite
       2. Handles threshold-bridging bids and settles all bidders at uniform clearing price:

      AssertionError: expected 599999999999998500000 to equal 600000000000000000000.
      + expected - actual

      -599999999999998500000
      +600000000000000000000

      at Context.<anonymous> (test\DutchAuction.test.js:49:61)
```

*Figure 1.1 — Our hand-written v1: 0 passing, 4 failing (audit/evidence/2026-10-22-baseline-test-run.txt).*

| # | Issue in v1 (our human baseline) | Severity | Found by |
|---|---|---|---|
| 1 | The price drifted because discountRate divides before it multiplies | High | Tests 1 and 2 |
| 2 | "Burning" sent tokens to address(0), which OpenZeppelin always rejects, so any auction with unsold tokens was bricked | Critical | Test 3, Slither |
| 3 | Reentrancy in claimTokensAndRefund: the attacker left with 8 ETH + 200 NTU instead of 4 ETH + 100 NTU | Critical | Test 4, Slither |
| 4 | The bidder who crossed the threshold was refunded twice (Bob paid 7 ETH net instead of 16) | Critical | Hidden; found by a probe |
| 5 | The seller's withdrawal swept address(this).balance, taking refunds owed to bidders | Critical | Hidden; found by a probe |

| | | | |
|---|---|---|---|
| 6 | The auction clock started at deployment, before the auction held any tokens | High | Hidden; found by a probe |
| 7 | **Spec flaw:** per-bid allocation contradicts the brief's clearing rule, and paid the seller 17% less | Critical | Manual review |
| 8–13 | Unchecked transfer, state written after the refund call, a missing event, rounding dust, push refunds, dead code | Medium to low | Slither, review |

The full analysis for review can be found in *audit/2026-10-22-issues-report.md* and *audit/evidence/2026-10-22-findings.md.* Slither flagged only issues 2, 3, 8 and 9. It missed 5 of the 7 major issues, because they were logic errors. That result shaped our whole testing strategy (Section 6).

## 2. Development Methodology & AI Tooling Infrastructure

### 2.1 Toolchain

| Layer | Tools and versions |
|---|---|
| AI coding agent | Claude Code (VS Code extension), model Claude Opus 5.5, in sessions 01 to 05 |
| AI assistant | Claude (claude.ai), in session 06: the final re-run, independent probes, and drafting this report and the demo materials from our logs and evidence |
| AI orchestration | Multi-agent workflows in session 04: a *code-reader* agent, a *reproducer* agent working on throwaway chains, a *skeptic* agent whose job was to refute the others, and a *strict-marker* agent that attacked proposals against the brief and the rubric |
| Other AI | The dutch-auction-token-launch branch was generated with an AI tool we did not record |
| Contracts | Solidity 0.8.24 (optimizer, 200 runs), Hardhat 2.29.1, OpenZeppelin Contracts 5 (ERC20Burnable, SafeERC20, ReentrancyGuard) |
| Frontend | React 19, Vite 8, TypeScript 5.9, wagmi 2.19.5, viem 2.57, RainbowKit 2.2.11, TanStack Query 5 |
| Validation | Mocha and Chai with hardhat-network-helpers, solidity-coverage, Slither 0.11.6, hardhat-gas-reporter, Vitest 5, a Puppeteer end-to-end test, manual mutation testing, differential probes |
| Automation | GitHub Actions (.github/workflows/ci.yml): contract tests and coverage, a Slither --fail-high gate, frontend tests and build, and the end-to-end UI test, on every push |
| Local deployment | npm run demo: a Hardhat node, the factory deployment, six seeded listings, and the Vite dev server |

### 2.2 Custom artefacts we built to steer and check the AI

We did not just chat with the tool. We built a small infrastructure around it so that its output could be checked.

| Artefact | Purpose | How it steered or checked the AI |
|---|---|---|
| spec.md (v1, v2, v3) | Formal specification for the AI | A table mapping each clause of the brief to a rule, invariants that tests enforce, and "Coding Rules for AI Tools" (Section 3.2) |
| prompts/ (6 session logs) | Prompt logs | Every prompt, what the AI did, our decisions and its mistakes. Section 3 draws on these |
| audit/ai-issues.md | Log of AI mistakes | 28 entries in the brief's four-part format, each with a prevention rule where one applies |
| audit/evidence/ | Evidence store | Raw logs, probe scripts and their output, screenshots. Each file is dated and never edited after capture |

| | | |
|---|---|---|
| Probe tests (evidence/2026-10-28-probes/, evidence/2026-11-01-review/independent-review.probe.js) | Differential and adversarial tests | The same scenario run against two implementations, and hypotheses tested on throwaway chains |
| Mutation runs (evidence/*mutations*) | A check on test quality | Each run plants one known bug. A planted bug that survives means a test that cannot fail |
| frontend/e2e/lifecycle.mjs | End-to-end test | Injects a test wallet, drives the real UI through the whole lifecycle, checks the bid gas limit, and fails if a page scrolls sideways at 390 px |
| tasks/auction.js, tasks/launchpad.js, scripts/demo.js | Automation | Deterministic deployment and seeding, so any claim the AI makes about behaviour can be replayed |
| CI workflow | Regression gate | AI changes cannot slip in if a test, the Slither gate or the end-to-end run fails |

### 2.3 Workflow

Each AI session followed the same loop:

1. **Specify:** We state the goal. The AI reads spec.md, the brief and the code before writing anything.
2. **Prototype in a scratch copy:** Prototypes were built outside the repository first. For example, the session 04 factory prototype passed 136 tests on a scratch copy before any of its code entered the repository.
3. **Challenge:** Tests, probes, mutation testing, Slither and screenshots. In session 04, a separate agent's only job was to refute the first agent's conclusion. In session 05, the AI audited its own earlier work from a fresh clone.
4. **Human gate:** We review the result, decide the scope and approve every commit. The AI pushed only when we asked it to.
5. **Log:** Prompts go to prompts/, mistakes to *ai-issues.md*, and raw output to *audit/evidence/*.

### 2.4 Division of labour and decision-making

| **Who** | **Role** |
|---|---|
| Syed Ridwan | Ran all five Claude Code sessions: set each session's goal, reviewed the output in the browser and in MetaMask, challenged the AI's claims, and approved commits. Kept the prompt logs and the AI issue log |
| Yojin Oh | Built the alternative implementation, dutch-auction-token-launch (Hardhat 3, viem, Ignition, six Sepolia deployments), with ChatGPT. It became our second design and the counter-example in our differential tests |

| | |
|---|---|
| Both of us | Wrote v1 by hand, agreed the scope decisions below, reviewed the AI's output before each commit, and planned the demo |
| Claude Code | Wrote code, tests, probes and documentation, and laid out options with their trade-offs. It never made a scope decision |

## 3. Specification Engineering & Prompt Logs

### 3.1 The specification changed four times, each time because of evidence

| Version | Author | What changed | Why |
|---|---|---|---|
| **v1** (*138f1ce*) | Us, by hand | Per-bid allocation, and refunds paid at claim time | Our original design |
| **v2** (session 01) | Claude Code, from the brief and v1 | Rewritten as a table mapping each clause of the brief to a rule. Committed-capital clearing, invariants, and coding rules for AI tools | v1's rule contradicted "when total committed capital meets total token supply, the auction clears", and paid the seller 17% less in our own test |
| **v2 corrections** (session 04) | Claude Code, after review | Two invariants restated: ETH forced in with selfdestruct, and dust bids smaller than 1 token-wei | Both were false as first written |
| **v3** (session 04) | Claude Code, within the scope we set | Section 6: the LaunchFactory rules, bounds, roles, the curator's limits, and known limits | We extended the scope to a launchpad without touching the audited auction |
| **v3 addition** (session 05) | Claude Code, after AI-28 | Section 5 rule: a function whose gas cost depends on when it is mined must not rely on a wallet's estimate | A sell-out bid could run out of gas |

The core of the spec is the table mapping the brief to rules (excerpt from *spec.md* section 2):

| Brief clause | Rule in our system |
|---|---|
| Descending price, predefined decay function | P(t) = P_start − (P_start − P_reserve) × t / 1200, multiplying before dividing, with t from block.timestamp |
| Clears when committed capital meets total supply | Sold out when totalCommitted ≥ S × P(t) / 1e18. Bids revert once sold out |
| A bid that bridges the threshold (tricky point) | Accepted up to room = S × P(t) / 1e18 − totalCommitted. The auction finalizes in the same transaction |
| Every winning bidder pays the same price | P_clear = max(P_reserve, totalCommitted × 1e18 / S). Each bidder gets commitment × tokensSold / totalCommitted |
| Excess ETH is refunded | The bridging bid's surplus is refunded inside bid(). This is the only refund path |
| Sell out at or above the reserve, or burn | On expiry, P_clear = P_reserve, and the rest is burned with ERC20Burnable.burn |

### 3.2 Coding rules we wrote for the AI

We wrote these rules (spec section 5) after reviewing v1, specifically to stop the AI repeating our v1 mistakes. Section 5 shows where the AI still broke them, and how the rules caught it.

- Multiply before divide in all fixed-point maths; units are wei per whole token. [*v1 issue 1*]
- Checks-effects-interactions; nonReentrant on ETH- or token-moving functions. [*v1 issues 3, 9; caught AI-21*]
- SafeERC20 for all token transfers; burn with ERC20Burnable.burn, never to address(0). [*v1 issues 2, 8*]
- Never send *address(this).balance*; pay only computed, tracked amounts. [*v1 issue 5*]
- Tests assert ETH and token balances, events, reverts and invariants. [*v1 issue 4; caught AI-22*]
- Gas that depends on when a tx is mined must not rely on a wallet's estimate. [*added after AI-28*]

### 3.3 The context we gave the AI in each session

| Session | Date | What was in the context window | Purpose |
|---|---|---|---|
| 01 | 22 Oct | The brief (PDF) and our repository at *ef71e56* (v1) | Diagnose v1, redesign, build tasks, frontend and CI |
| 02 | 28 Oct | The repository at *9c89440* | A guided demo after a break, and one UI fix |
| 03 | 28 Oct | The repository at *198e641*, and the dutch-auction-token-launch branch (*792b50d*) | Compare the two implementations and combine them |
| 04 | 30 Oct | The repository at *1ed3694*, the brief, the submission instructions (pasted) and screenshots | Diagnose MetaMask, test a scope change against the rubric, build the launchpad, redesign |
| 05 | 31 Oct | The repository at *81940cd*, and the brief | Check the whole project against the brief, and plan the demo |
| 06 | 31 Oct – 1 Nov | The brief, and the project ZIP before and after session 05 | Final re-run and independent probes, this report, the demo script and slides |

We put screenshots into the context several times (S02 P4, S03 P5–7, S04 P1 and P8). Each time, we had seen a problem in the running app that no automated test had flagged.

### 3.4 Representative prompt chains

The quotes are taken word for word from prompts/. Each chain shows an iteration: our prompt, the AI's output, our challenge, and the correction.

**Chain A: From failure to redesign (session 01, prompts 1 and 2).**

> "I am having some trouble with my blockchain project (we are doing option 2 - dutch auction). Please run and analyse my current project and compile a report of the current issues in testing (explain them and why they matter). Then, analyse those issues and suggest fixes and provide alternatives. [...] (adhere to rubrics strictly)."

The AI ran the suite (0 of 4 passing), patched a scratch copy one fix at a time to isolate each root cause, wrote probes that exposed three hidden money bugs, and set out Option A against Option B. We replied:

> "I want to adhere to the brief strictly (the dutch auction)"

The result was spec v2, a rewritten contract, 29 tests and mutation testing, which caught a rounding bug in the AI's **own** code (AI-1).

**Chain B: Differential comparison (session 03, prompt 1).**

> "Please conduct a thorough analysis of the branch named "dutch-auction-token-launch" and compare this current branch with that one. Compare and analyse where the strengths are and complement each other to combine into one perfect solution and create it on a new branch named "soln-combined""

The AI ran the same bids against both contracts (probes P1 to P3). The per-bid branch paid the seller **166.67 ETH** where the committed-capital contract paid **600 ETH** (Figure 5.3). It kept the better contract and ported the other branch's token tests (AI-10 to AI-14).

**Chain C: From symptom to root cause (session 03, prompts 6 and 7).**

> "Why do I keep getting these?" *(with a screenshot of failing CI runs #3 and #6)*

The AI read the run results, asked us for the failing step's log, and traced an intermittent timeout to a race in the transaction hook (AI-17).

**Chain D: Testing a scope change against the rubric (session 04, prompts 2 and 6).**

> "I want it to be a fully functional site where there are multiple coins available for bidding, different user accounts (e.g. user, seller, buyer, admin etc.). It would be nice if sellers could create their own tokens just from the website. [...] please evaluate if this still adheres to the rubrics."

A multi-agent workflow prototyped a factory (136 tests on a scratch copy), and a strict-marker agent attacked the proposal. Its verdict: the idea fits Option 2 **only as a launchpad**, not as an "exchange" with accounts and a general admin. We accepted that, and the AI built it in three committed stages, with a fallback tag (fallback-single-auction).

**Chain E: The human eye catches what tests cannot (session 04, prompts 7 and 8).**

> "The overall UI looks very basic and AI-generated. It must be more dynamic and not "containerised" looking. i want a light theme too"
>
> "The auction has ended but the numbers in "under the hood" keep changing"

These led to a full redesign (AI-23) and a fix to the price display (AI-25). No automated test could see either problem.

**Chain F: Making the AI audit itself (session 05, prompt 1).**

> "Please run through my entire project and refer to the attached assignment brief to ensure all the criteria are met to secure an A+ and stand out from the rest."

The AI cloned the branch fresh, followed the README as a marker would, and walked our video plan through the real pages. It found three mistakes in its own earlier output that 120 passing tests had not shown: a seed command that crashed (AI-26), a false "at least" in the bid preview (AI-27), and a bid that could run out of gas (AI-28).

## 4. AI-Generated Architecture

### 4.1 System overview

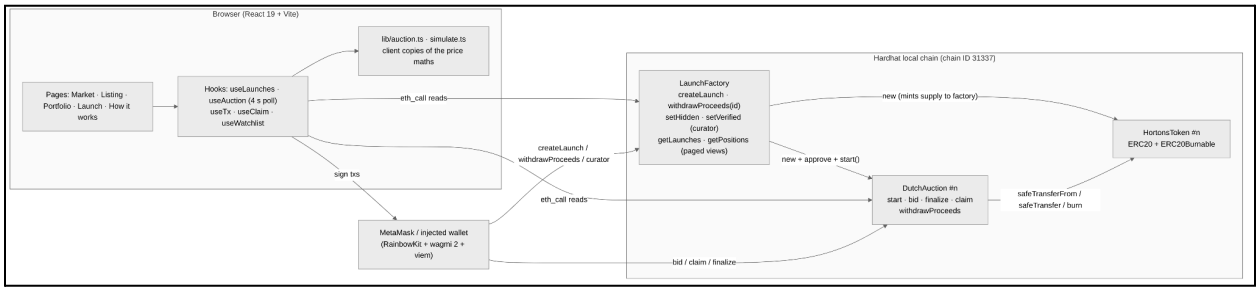



*Figure 4.1 — The system architecture the AI proposed in sessions 01 and 04, which we accepted.*

The AI proposed three layers: one factory; one token and one auction per listing; and a React app with no backend, which reads everything from the chain.

### 4.2 Contract interfaces

DutchAuction (session 01, unchanged since). One auction, with immutable parameters and no loops.

| Function | Who can call it | What it does |
|---|---|---|
| constructor(token, totalTokens, startPrice, reservePrice) | The seller | Validates the parameters, and sets seller = msg.sender |
| start() | The seller, once | Pulls totalTokens with safeTransferFrom, then starts the 20-minute clock, so the auction can never run unfunded |
| currentPrice(), soldOut(), isActive() | Anyone (view) | A stateless price, computed from block.timestamp on each call |
| bid() — payable, nonReentrant | Anyone | Records ETH only. Caps the bridging bid at room, then finalizes and refunds the surplus in the same transaction |
| finalize() | Anyone | Settles the auction after a sell-out or expiry |
| claim() — nonReentrant | Each bidder, once | Finalizes first if needed, then pays a pro-rata share of tokensSold |
| withdrawProceeds() — onlySeller, nonReentrant | The seller, once | Pays exactly totalCommitted, never address(this).balance |
| _finalize() — internal | — | The one path for every settlement. Fixes clearingPrice and tokensSold, burns the rest, and emits AuctionFinalized |

**LaunchFactory** (session 04)

| Function | Who can call it | What it does |
|---|---|---|
| createLaunch(name, symbol, supply, startPrice, reservePrice) | Anyone | Checks the bounds, deploys a token and an auction with new, then approves and calls start(). All in one transaction |
| withdrawProceeds(id) — nonReentrant | The listing's registered seller | The factory is the on-chain seller, so it withdraws and forwards exactly totalCommitted |
| setHidden(id, bool), setVerified(id, bool) | The curator only | Display flags. No auction reads them |
| getLaunches(offset, limit), getPositions(account, offset, limit) | Anyone (view) | Pages of up to 50 rows for the market and portfolio pages, so there is no backend or indexer |
| receive() | Its own auctions only | Rejects ETH from anyone else |

**HortonsToken**: a fixed-supply ERC20 with ERC20Burnable, minted once to its deployer. It has no owner and no mint function.

## 4.3 State management

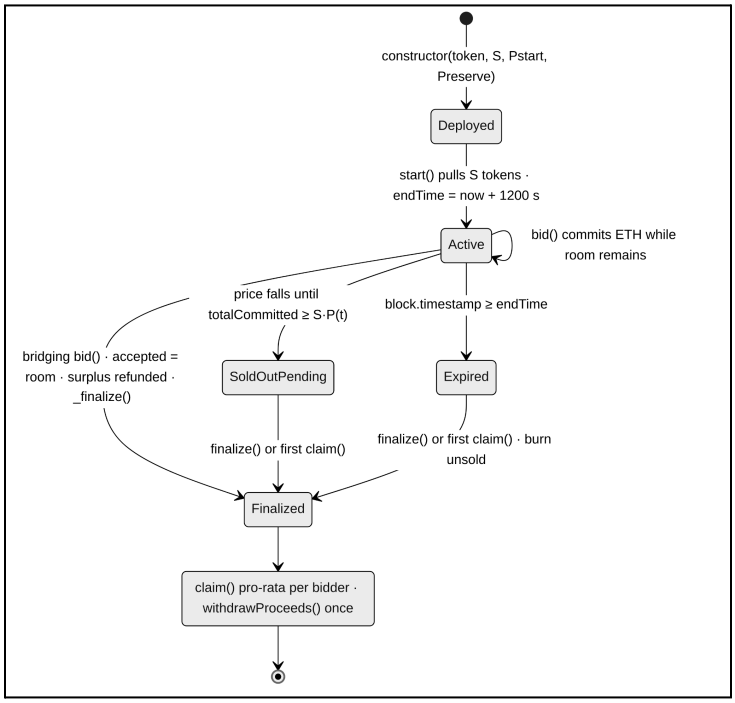



*Figure 4.2 — The DutchAuction lifecycle. Note the "SoldOutPending" state: a falling price can sell an auction out between transactions, and anyone can then settle it.*

| Layer | State | Design choice the AI proposed |
|---|---|---|
| On-chain, per auction | startTime, endTime, totalCommitted, commitments[], claimed[], clearingPrice, tokensSold, finalized, proceedsWithdrawn | The price is **not stored**. It is derived from the time, so there is nothing to update and no drift |
| On-chain, factory | The Launch[] registry, the _isAuction set, and an immutable curator | A registry only. The factory holds no funds between transactions |
| Frontend, chain data | useLaunches and useAuction (TanStack Query, polled every 4 seconds) | The chain is the single source of truth. There is no database |
| Frontend, derived values | lib/auction.ts, lib/simulate.ts | Client copies of the contract's maths update the price every second between polls. Unit tests check them against the contract's checkpoints |
| Frontend, per viewer | useWatchlist (browser storage) and the theme | Conveniences only. They never affect funds |
| Roles | Worked out from the connected wallet and chain data | No accounts or logins. The seller created a listing, a bidder has a commitment, and the curator is one fixed address |

### 4.4 Design alternatives the AI evaluated

| Decision | Alternatives measured | Choice and reason |
|---|---|---|
| Allocation | Per-bid (our v1, and YJ's branch) or committed capital | Committed capital: the brief's wording. Probe P1 shows that per-bid allocation fails to clear |
| Factory deployment | EIP-1167 minimal clones or plain new | Clones made each launch **3.4 times cheaper**, but needed 35 added and 7 removed lines in the audited auction, plus a new token. Plain new leaves the audited bytecode unchanged, and a test compares it byte for byte |
| Token input | Any ERC20, or factory-made tokens only | Factory-made only. In the prototype, 5 of 5 unusual tokens misbehaved, and a non-burnable one locked ETH |
| Refunds | Pull refunds (withdrawRefund) or a same-transaction refund | A same-transaction refund of the bridging surplus, which is the only surplus under committed capital |
| Curator powers | Pause and upgrade, or display flags only | Display flags only. We removed the prototype's pause power |
| Bid gas limit (session 05) | The wallet's estimate, or a fixed limit | A fixed limit of 200,000. The estimate can be for the wrong code path (AI-28) |

## 5. Problems Discovered & Vulnerability Analysis

### 5.1 Inventory of AI mistakes

Severity scale. *High*: funds lost or locked, a broken build, or a contradiction of the brief. *Medium*: wrong behaviour, a misleading display or false confidence from a test, with no funds lost. *Low*: cosmetic, quickly corrected by the AI itself, or documentation only. *(Process)* marks mistakes in the audit material itself.

| # | Issue | Category in the brief | Severity | Session | Status |
|---|---|---|---|---|---|
| AI-1 | Claims over-allocate when the clearing price rounds down | Integer/math | **High** | 01 | Fixed |
| AI-2 | Stated an unverified guess (that v1 was AI-written) as fact | Hallucination (process) | **High** | 01 | Fixed |
| AI-3 | Wrong arithmetic in the Slither timestamp triage | Integer/math (docs) | Low | 01 | Fixed |
| AI-4 | An append to .gitignore merged two rules | Tooling | Low | 01 | Fixed |
| AI-5 | Blamed the wrong library for a crash on exit | Misdiagnosis | Low | 01 | Fixed |
| AI-6 | Status output claimed a false "guaranteed minimum" | Logic | Medium | 01 | Fixed |
| AI-7 | The end-to-end test reported "connected" while the wallet was still connecting | Front-end provider state; false test | Medium | 01 | Fixed |
| AI-8 | An unreadable, overlapping chart on mobile. The first fix made it worse | UX | Medium | 01 | Fixed |
| AI-9 | Package versions written from memory that did not exist | Hallucinated versions | Low | 01 | Fixed |
| AI-10 | Per-bid allocation contradicts the brief's clearing rule | Logic; price calculation | **High** | 792b50d | Not carried over |
| AI-11 | @openzeppelin/contracts missing, so a fresh clone does not compile | Build | **High** | 792b50d | Not carried over |
| AI-12 | The clock starts at deployment, so an unfunded auction takes bids and then locks the ETH | State; stuck funds | **High** | 792b50d | Not carried over |
| AI-13 | Seller proceeds grow only as bidders claim | Logic; stuck funds | Medium | 792b50d | Not carried over |

| | | | | | |
|---|---|---|---|---|---|
| AI-14 | Template code left in, and mismatched hard-coded addresses | Front-end provider config | Low | 792b50d | Not carried over |
| AI-15 | Misread commit ancestry from a git log --graph listing | Misreading | Low | 03 | Fixed |
| AI-16 | Called a branch "the team's" without confirming it (a repeat of AI-2) | Hallucination (process) | Medium | 03 | Fixed |
| AI-17 | A race in the transaction hook wiped typed input, and made CI flaky | Front-end state | Medium | 01, found in 03 | Fixed |
| AI-18 | Offered an untested theory for a false MetaMask error | Misdiagnosis | Low | 04 | Fixed |
| AI-19 | An agent claimed MetaMask never reuses a confirmed nonce, from a truncated read | Hallucination from partial context | Medium | 04 | Fixed |
| AI-20 | The factory's bounds allowed a flat "descending" price and instant dust sell-outs | Logic; price calculation | **High** | 04 | Fixed before merging |
| AI-21 | No nonReentrant on the factory's ETH forward | Reentrancy | Medium | 04 | Fixed |
| AI-22 | Tests that could not fail: a swallowed assertion, and tests blind to their own mutation | False test confidence | Medium | 04 | Fixed |
| AI-23 | A generic-looking interface | UX | Medium | 04 | Fixed |
| AI-24 | Hidden tooltips made pages scroll sideways on phones | UX | Low | 04 | Fixed |
| AI-25 | The displayed price kept falling after a sell-out, contradicting the clearing price | Price calculation (display) | Medium | 04 | Fixed |
| AI-26 | The seed task accepted expiry values that crashed it, and the video plan used one | Logic; tooling | Medium | 05 | Fixed |
| AI-27 | The sell-out preview promised "at least" more tokens than the bidder received (a repeat of AI-6) | Price calculation (display) | Medium | 05 | Fixed |
| AI-28 | A bid estimated as a plain bid could be mined as a sell-out and run out of gas | **Unhandled gas limits** | Medium | 05 | Fixed |

**Totals:** 28 mistakes: 6 high, 14 medium and 8 low. Claude Code made 23 of them across our sessions. The unrecorded AI tool behind dutch-auction-token-launch made the other 5.

**How they were caught:**

- Tests or mutation testing: AI-1, AI-7, AI-17, AI-21, AI-22 and AI-24.
- The AI's own self-review: AI-3, AI-4, AI-6, AI-16, and part of AI-22.
- An adversarial agent: AI-18, AI-19 and AI-20.
- Differential probes: AI-10 to AI-13.
- The AI auditing its own work from a fresh clone: AI-26, AI-27 and AI-28.
- Us: AI-2, AI-23 and AI-25, and the CI failure that led to AI-17.

Four of the sixteen medium and high issues from our own sessions needed one of us to notice them.

The full breakdown of every issue is in audit/ai-issues.md. The major ones follow in the brief's format.

### 5.2 Major issue breakdowns (the brief's section 2.2 format)

AI-1 — Claims over-allocate when the clearing price rounds down (High · integer/math · stuck tokens)

**Why the AI made the mistake:** In its v2 prototype, the AI computed each bidder's tokens as commitment × 1e18 / clearingPrice, assuming the shares would add up to no more than the supply. But clearingPrice is itself rounded *down*, so each share comes out slightly too large. The AI then checked the prototype with three tests that used only round numbers, where no rounding happens, and reported "3 of 3 passing" as evidence.

**Why it is problematic:** When an auction sells out because the price fell, the claims add up to more than the contract holds. The last bidder's claim() reverts with ERC20InsufficientBalance, and their tokens are stuck forever. In the test, the shortfall was 2,263 token-wei.

**How we fixed it:** Claims became a pro-rata share of what was actually sold, which can never exceed tokensSold:

```
// before (AI prototype)
        return (commitments[bidder] * TOKEN_UNIT) / clearingPrice;
 // after (contracts/DutchAuction.sol, tokensOf)
        return (commitments[bidder] * tokensSold) / totalCommitted;
```

We also added a targeted test with odd bid amounts: *"never pays out more than tokensSold when the clearing price rounds down"*.

**Evidence the fix works:** Mutation M7, which puts the old formula back, **survived all 28 tests** before the targeted test existed, and is caught afterwards

(evidence/2026-10-22-findings.md, mutation table). The test passes in the final re-run (Figure 6.1), and ten seeded random auctions assert that the claims never add up to more than tokensSold.

**AI-10 — Per-bid allocation contradicts the brief's clearing rule (High · logic · inaccurate price calculation)**

**Why the AI made the mistake:** In dutch-auction-token-launch, bid() fixes each bid's tokens at msg.value × 1e18 / getCurrentPrice(), and counts the auction as sold out only when those fixed quantities reach the supply. This "buy N tokens at today's price" model is common in auction tutorials, and is easy to confuse with a uniform-price auction. The tool had the brief's wording, but did not turn "when total committed capital meets total token supply, the auction clears" into code. Our own v1 made the same mistake, which is why our spec v2 quotes the brief clause by clause.

**Why it is problematic:** Strong demand can fail to clear the auction, and the seller is underpaid. Probe P1 used 1,000 tokens falling from 1 to 0.2 ETH, with Alice committing 400 ETH at 5 minutes and Bob 200 ETH at 10 minutes. The per-bid contract **never sells out**: it expires at the 0.2 ETH reserve, burns 166.67 tokens and pays the seller **166.67 ETH**. Under the brief's rule, the same bids sell out at 0.6 ETH and pay **600 ETH**.

**How we fixed it:** We did not carry this design into soln-combined, which keeps the committed-capital contract. P1 became a permanent regression test: *"sells out on committed capital where per-bid allocation would settle at the reserve"*.

**Evidence the fix works:**

audit/evidence/2026-10-28-probes/*.output.txt · differential probes, same bids on both contracts

```
$ npx hardhat test …/committed-capital-contract.probe.js
Compiled 16 Solidity files successfully (evm target: paris).
  PROBE committed-capital contract
finalized in Bob's tx: true
clearing 0.6 | seller gets 600.0 | supply left 1000.0 | Alice NTU 666.666666666666666666 | Bob NTU 333.333333333333333333
    ✔ P1 strong demand: Alice 400 ETH at t=300, Bob 200 ETH at t=600 (1425ms)
  1 passing (1s)

$ npx hardhat test nodejs test/per-bid-contract.probe.ts   # on branch dutch-auction-token-launch (792b50d)
Running node:test tests
soldOut after 600 ETH at P=0.6: false
clearing 0.2 | seller proceeds 166.666666666666666666 | supply left 833.333333333333333333 | contract ETH dust (wei) 0n
  PROBE yj contract
    ✔ P1 strong demand that never fills by quantity (4541ms)
seller blocked until Alice claims: confirmed
    ✔ P2 seller cannot withdraw while bidders have not claimed
unfunded bid accepted; ETH held: 100
finalize and claim both revert: Alice's 100 ETH is locked
    ✔ P3 unfunded auction accepts bids, then cannot settle
  3 passing (6522ms)
```

*Figure 5.3 — The same bids on two contracts. Committed capital sells out at 0.6, and the seller gets 600 ETH. Per-bid allocation clears at 0.2, and the seller gets 166.67 ETH. P2 shows the seller blocked until bidders claim (AI-13), and P3 an unfunded auction locking 100 ETH (AI-12). Source: evidence/2026-10-28-probes/.*

**AI-12 — The clock starts before funding, and ETH is locked for good (High · state across transactions · stuck funds)**

**Why the AI made the mistake:** The generated constructor set startTime = block.timestamp, and the Ignition module sent the tokens in a *separate* transaction afterwards. The AI treated "deploy" and "fund" as one moment. Nothing checked that the auction held the supply before accepting bids.

**Why it is problematic:** In probe P3, an unfunded auction accepted a 100 ETH bid. After expiry, finalizeAuction reverts because it cannot burn tokens it does not hold, and claim reverts because the auction never finalizes. **The ETH is locked forever.** Even when the auction is funded, the 20-minute window starts before the funding transaction lands.

**How we fixed it:** We did not carry it over. Our start() pulls the supply *before* starting the clock, and bids revert before start():

```
function start() external onlySeller {
    require(startTime == 0, "Already started");
    startTime = block.timestamp;
    endTime = block.timestamp + DURATION;
    emit AuctionStarted(startTime, endTime);
     IERC20(address(token)).safeTransferFrom(msg.sender, address(this), totalTokens);
// reverts the whole start if unfunded
 }
```

In the launchpad, createLaunch deploys, approves and starts in **one transaction**, so the gap cannot exist at all.

**Evidence the fix works:** The P3 output (Figure 5.3); the tests *"does not accept bids before start"* and *"start() is seller-only, single-use and requires token approval"*; and the factory test *"moves the whole supply into a running auction and leaves the factory with nothing"* (Figure 6.2).

**AI-13 — Seller proceeds depend on bidders claiming (Medium · logic · stuck funds)**

**Why the AI made the mistake:** sellerProceeds grew only inside claim(), by each claimer's cost. The AI tied two independent parties' payments together.

**Why it is problematic:** A single bidder who never claims holds back the seller's payment for their tokens forever. In probe P2, the seller's withdrawal reverts with "No proceeds available" after the auction has ended.

**How we fixed it:** We did not carry it over. Our seller withdraws exactly totalCommitted, once, whether or not bidders have claimed.

**Evidence the fix works:** P2 (Figure 5.3); the test *"pays the seller exactly the committed ETH, once, without touching bidders' tokens"*; and mutation M3 (removing the proceedsWithdrawn check) is caught.

**AI-20 — Weak factory bounds: a flat "descending" price and instant dust sell-outs (High · logic · price calculation)**

**Why the AI made the mistake:** In the factory prototype, the AI set each bound only to stop a revert or an overflow: reservePrice > 0, and a minimum decay of 1,200 wei. It tested whether the code *ran*, not whether the result still met the brief. Its fuzz test also clustered reserves at high values, so low prices were rarely tried.

**Why it is problematic:** A start price of reserve + 1200 wei is a flat-price sale, against "descending price". A 1-token listing at 1 gwei sells out on the first bid, seconds after it is created, against the spirit of "exactly 20 minutes". And a 1-wei reserve misreported the clearing price by 50%.

**How we fixed it:** A separate *strict-marker* agent read the prototype against the brief, and flagged these problems before any of the code reached our repository. The real factory enforces:

```
require(reservePrice >= MIN_RESERVE, "Reserve too low"); // 1 gwei
 require(startPrice >= reservePrice * MIN_PRICE_RATIO, "Start must be at least 2x reserve");
 require((supply * reservePrice) / 1e18 >= MIN_RAISE, "Raise too small"); // 0.001 ETH
```

The Launch form repeats the same bounds (frontend/src/lib/launch.ts), so a user sees the error before their wallet opens. The fuzz test now spreads reserves across the whole range.

**Evidence the fix works:** The 14 "Parameter bounds" tests pass (Figure 6.2). Mutations M6 to M8, which each remove one bound, are caught (Figure 6.6). The frontend tests *"rejects a nearly flat curve"* and *"rejects a raise under 0.001 ETH"* pass (Figure 6.4), as does the end-to-end step *"form explains invalid values before the wallet opens"* (Figure 6.5). Probe R2 (Section 5.4) confirms that the factory rejects the inputs the prototype accepted.

**AI-21 — No reentrancy guard on the factory's ETH forward (Medium · reentrancy)**

**Why the AI made the mistake:** The prototype's withdrawProceeds(id) forwarded ETH to the seller without nonReentrant. Its re-entry test passed anyway, because the auction's own "Already withdrawn" check blocked a replay of the *same* launch. So the

AI concluded the guard was unnecessary. It tested one attack path and generalised from it.

**Why it is problematic:** It broke our own spec's rule (section 5: nonReentrant on every function that moves ETH). A seller contract with two launches could re-enter from its receive() to withdraw the second launch in the middle of the first. That is exactly the cross-function reentrancy pattern the brief's bonus is about.

**How we fixed it:** We added nonReentrant to LaunchFactory.withdrawProceeds, and a test in which a malicious seller contract (ReentrantSeller in contracts/LaunchTestHelpers.sol) re-enters for a *different* launch.

**Evidence the fix works:** Mutation M2, which removes the guard, **survived** the original suite with 0 tests failing. With the new test added, 1 test fails (Figure 6.6). The tests *"a seller cannot re-enter the factory to withdraw a second launch mid-withdrawal"* and *"a reentrant seller collects once and cannot reach a neighbour's ETH"* pass (Figure 6.2).

**AI-22 — Tests that could not fail (Medium · false test confidence)**

**Why the AI made the mistake:** It could not know the new contract addresses in advance, so it wrapped an event assertion in .catch(() => {}). It gave a dust assertion escape clauses so that it would pass. And its page-size test used exactly the cap's size (50), so the cap was never applied. The AI optimised for green tests, not for tests that can tell right from wrong.

**Why it is problematic:** A test that passes for the wrong reason gives false confidence. That is the exact failure this audit exists to catch, and it repeats AI-7.

**How we fixed it:** The AI caught the first two on self-review, and rewrote them with exact values. Mutation testing caught the third (M13), and the test now creates 51 listings.

**Evidence the fix works:** M13 survived with 0 tests failing, and fails 1 test after the fix (Figure 6.6). *"pages through 51 listings and clamps the page size to 50"* passes.

**AI-28 — A sell-out bid could run out of gas (Medium · unhandled gas limits)**

**Why the AI made the mistake:** BidForm left the gas limit to the wallet. But bid() costs about 67k gas as a plain bid, and about 153k when it sells the auction out: it then also caps the bid, finalizes, burns and refunds. A wallet estimates the gas *before* its owner confirms, and the price keeps falling meanwhile, so a bid can be estimated as one path and mined as the other. None of our tests could show this. The injected test wallet sends no gas limit, so the local node gave every transaction 16.7 million gas.

**Why it is problematic:** A bid close to the sell-out line fails, the auction is unchanged, and the bidder still pays for the gas. No funds are at risk, but this is the "unhandled gas

limits" issue the brief names. It is also most likely to happen on the bid a demo cares about most: on Meridian, 30 seconds spent in the wallet removes about 2 ETH of room.

**How we fixed it:** The site now sends a fixed limit of 200,000 gas with every bid. Unused gas is returned to the bidder, so the higher limit costs nothing. spec.md section 5 records the rule:

```
// frontend/src/lib/auction.ts
 export const BID_GAS_LIMIT = 200_000n;
 // frontend/src/components/BidForm.tsx
  writeContractAsync({ address: listing.auction, abi: dutchAuctionAbi, functionName: "bid", value, gas: BID_GAS_LIMIT, chainId })
```

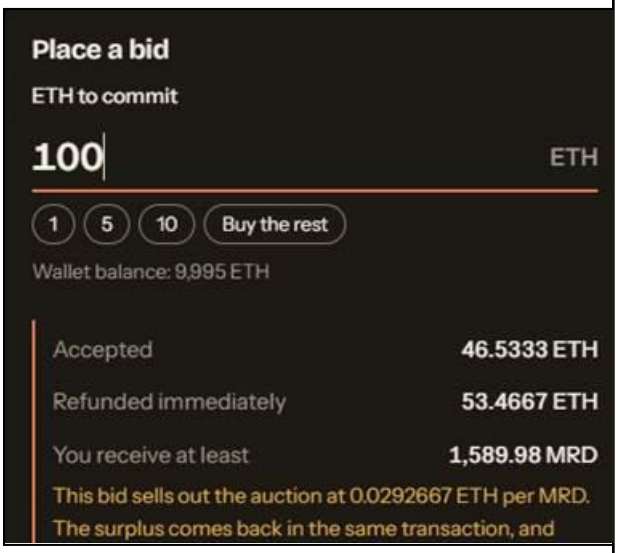
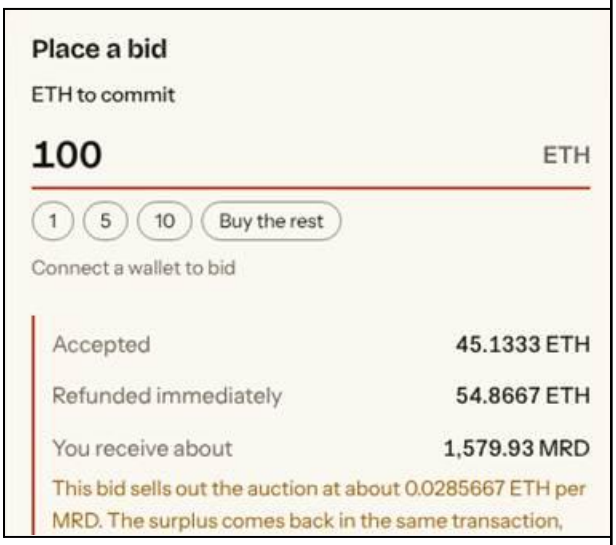
**Evidence the fix works:** A probe with limits of 100%, 115%, 120%, 150% and 200% of the wallet's estimate ran out of gas every time. Two tests in test/LaunchFactory.test.js, under "Gas limit for bids", keep that result and show that 200,000 is enough (Figure 6.2). The gas report puts the most expensive bid at 152,993 gas (Figure 6.7). The end-to-end step *"20 ETH bid confirmed, with a gas limit that covers the sell-out path"* now checks the limit, and fails with *"Bid sent without the fixed gas limit"* if it is removed (Figure 6.5).

**AI-27 — A minimum that was not a minimum (Medium · inaccurate price calculation, display)**

**Why the AI made the mistake:** The bid preview used one formula, tokens at the current price, for every bid. That is a true minimum for a plain bid. But a sell-out bid is capped at the remaining room, and the room shrinks as the price falls. By the time the bid is mined, the bidder's accepted ETH and their share are smaller. This is the same reasoning error as AI-6, in new code.

**Why it is problematic:** The site promised a bidder something false. In a run through our video plan, the preview said "You receive at least 1,589.98 MRD", and the claim paid 1,589.51 MRD. The brief names inaccurate price calculation directly. The contract paid the correct amount; only the wording was wrong.

**How we fixed it:** A sell-out preview now reads "You receive about", and its note says "at about" that price. A plain bid keeps "at least", which still holds.

| Before the fix | After the fix |
| --- | --- |
|  |  |
| "You receive **at least** 1,589.98 MRD". The claim paid 1,589.51 | "You receive **about** 1,579.93 MRD", "at about 0.0285667 ETH" |

**Evidence the fix works:** The before-and-after screenshots in *evidence/2026-10-31-ui/*, and a fresh capture from the final build can be located in *evidence/2026-11-01-review/demo-screenshots/*. *frontend/e2e/lifecycle.mjs* now waits for the new wording on a sell-out bid.

**AI-17 — A race in the transaction hook wiped typed input and made CI flaky (Medium · front-end state)**

**Why the AI made the mistake:** useTx.run set the "confirmed" status, *then* waited for every query to refresh, and only then returned true, which is when BidCard clears its input. The AI treated the success message and the form reset as one moment, but a network round trip separated them.

**Why it is problematic:** Anything typed between "Bid confirmed" and the end of the refresh was wiped. CI failed intermittently: runs #3 and #6 timed out waiting for *"This bid sells out the auction"*. A real user typing a quick second bid would hit the same bug.

**How we fixed it:** The hook now refreshes first and reports success afterwards, so the message and the reset happen in the same render:

```
const receipt = await client.waitForTransactionReceipt({ hash });
 if (receipt.status !== "success") throw new Error("Transaction reverted");
 await queryClient.invalidateQueries(); // moved before the success status
 setStatus({ state: "success", label, hash });
```

**Evidence the fix works:** The CI run #6 log shows the failure. After the fix, the end-to-end test passed 3 of 3 times locally, and the final re-run passes all 17 steps. These include *"100 ETH bid capped, surplus refunded, auction settled"*, which types the second bid immediately after "Bid confirmed" (Figure 6.5).

**AI-25 — The displayed price kept falling after a sell-out (Medium · inaccurate price calculation, display)**

**Why the AI made the mistake:** The "Under the hood" panel and the mini-charts mirrored currentPrice(), which keeps falling for the full 20 minutes even after the auction has sold out. The AI checked the panel on a live auction only. Its tests never looked at an ended auction that had sold out early, and the new code did not reuse the chart's existing logic.

**Why it is problematic:** A settled page contradicted itself. Meridian showed "Price now 0.0239667 ETH" under a 0.0263333 ETH clearing price, and the number was still ticking. The brief names "inaccurate price calculation" explicitly. The contract's payouts were correct; this was display only.

**How we fixed it:** One of us spotted it. One shared function now freezes the schedule at the sell-out, or at 20 minutes, and every page uses it:

```
export function scheduleElapsed(params, state, now) {
  const elapsed = now > state.startTime ? now - state.startTime : 0n;
   const stop = sellOutOffset(params, state.totalCommitted) ?? DURATION; // freeze at the sell-out
  return elapsed < stop ? elapsed : stop;
}
```

**Evidence the fix works:** The unit tests *"stops at the sell-out, however much later it is read"* and *"stops at 20 minutes if the auction never sells out"* pass (Figure 6.4). Figure 5.4, captured from the final build, shows a settled listing whose panel reads **"Price when it ended"**, computed at 602 seconds, which equals the clearing price exactly.

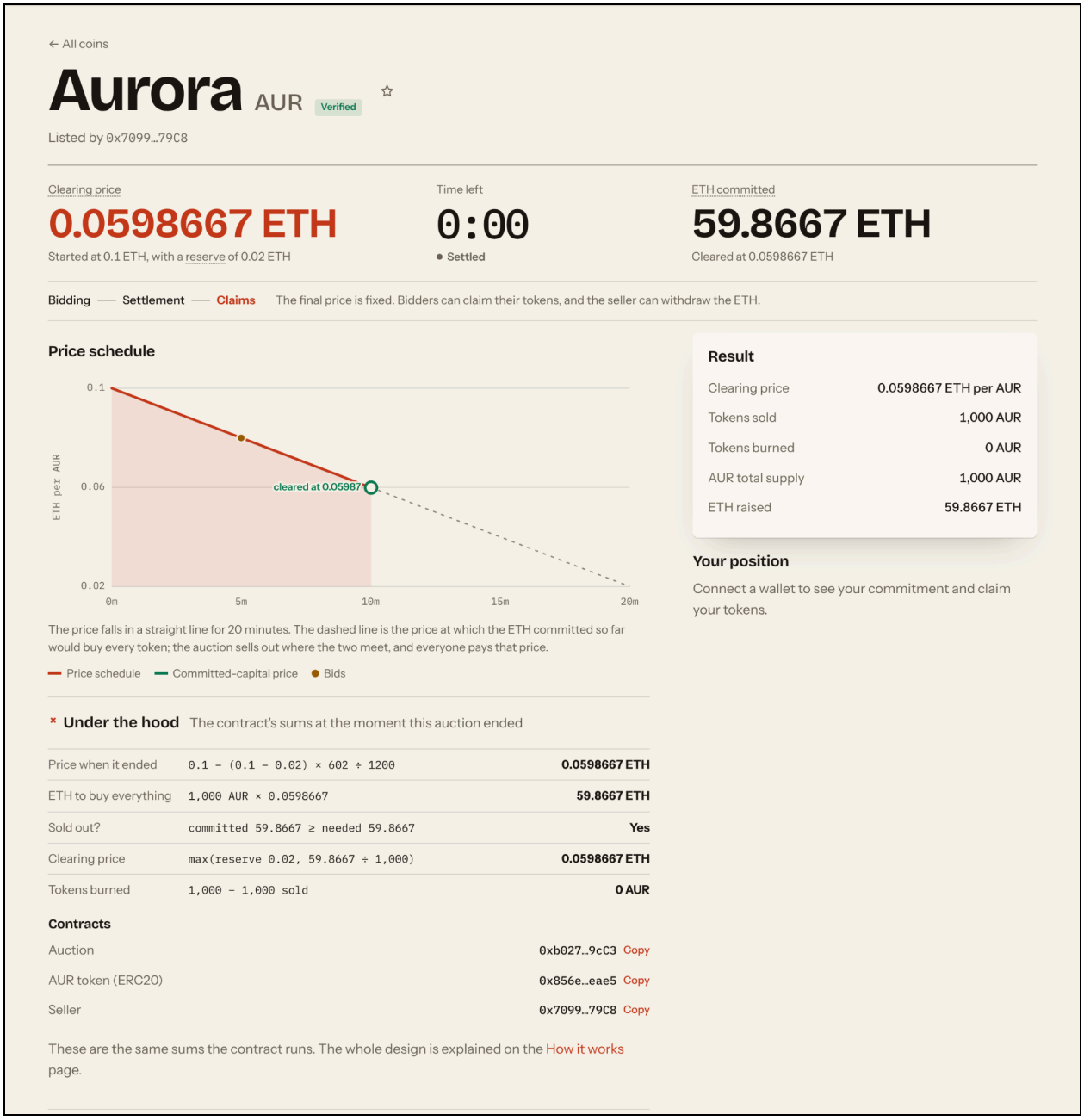



*Figure 5.4 — Aurora, sold out at 602 seconds and settled. "Price when it ended" equals the clearing price, 0.0598667 ETH. Committed 59.8667 ETH equals the 59.8667 ETH needed, and nothing was burned.*

## AI-2 and AI-16 — Unverified authorship stated as fact (High and Medium · process hallucination)

**Why the AI made the mistake:** It inferred that v1 was AI-written from the brief's framing (an audit of AI code), from commit timing (three commits in 11 minutes) and from code patterns. None of these shows how code was written. It then wrote the guess into the report as fact ("the AI copied…", "a hallucinated idiom"). In session 03 it repeated the

pattern, calling the other branch "the team's second implementation" because of the repository name and the commit author.

**Why it is problematic:** This report is assessed work. Attributing our hand-written code to AI would misrepresent our work, and misframe Sections 1, 4 and 5, which would undermine the credibility of the whole audit.

**How we fixed it:** We challenged the claim (Chain B). The AI traced every mention, rewrote the root causes neutrally, renamed the git tag, and recorded provenance at the top of the spec, the issue log and this report. We added a standing rule: **facts about provenance, intent or authorship come from us, never from inference.**

**Evidence the fix works:** The revision history of the issues report (revisions 24 to 50, snapshot in 2026-10-22-issues-report.md), and the session logs for 01 (prompts 3 and 4) and 03 (prompts 1 and 2). The AI caught AI-16 on its own self-review before committing, so the rule worked the second time.

### AI-19 — A confident claim from a truncated read (Medium · hallucination from partial context)

**Why the AI made the mistake:** While diagnosing a false "Transaction failed" message, a code-reading agent read minified MetaMask code that showed Math.max over local confirmed nonces. It concluded that old records raise the next nonce. But it had seen the filter only in truncated form: in the bundle the arguments are swapped, so that list is always empty.

**Why it is problematic:** The claim would have ruled out the real cause, which is nonces colliding with records from before a chain restart, and sent the diagnosis the wrong way.

**How we fixed it:** The *skeptic* agent ran the real bundled filter function on made-up records, and got 0 records where 1 existed, which overturned the claim. The final diagnosis, at about 75% confidence, led to the README instruction to clear MetaMask's activity after every chain restart.

**Evidence the fix works:** The session 04 log, prompt 1. After following the instruction, we reported "metamask works now" (S04 P6). The diagnosis was not reproduced end to end inside a real extension.

### AI-9 — Package versions written from memory (Low · hallucinated versions)

The AI wrote @types/react 19.2.2 and vitest 4.1.4 from memory. This is the "hallucinated versions or syntax" failure the brief names, and the install would have failed or pulled the wrong packages. It then checked every package with npm view before installing, and corrected them to 19.3.0 and 5.0.5. The same check showed that

RainbowKit 2.2.11 supports only wagmi 2, so wagmi is pinned to 2.19.5. Evidence: frontend/package.json, package-lock.json, and a clean npm ci in the final re-run.

### 5.3 Where each vulnerability class named in the brief stands

| Class (brief section 2.1.5) | In our v1 | In the AI's output | Final state | Evidence |
|---|---|---|---|---|
| **Reentrancy** | Exploitable: the attacker doubled its payout | AI-21: no guard on the factory's forward | nonReentrant on bid, claim and both withdrawProceeds. Checks-effects-interactions everywhere. Four reentrancy attack tests | Figures 6.1 and 6.2; M2 and M5 caught |
| **Access control** | The seller could drain the contract by sweeping its balance | The curator prototype had a pause power, since removed | onlySeller, a registered-seller check, and a curator limited to two display flags. An execution-trace test shows that curator calls make no external call | Figure 6.2; M1, M4, M5 and M9 caught |
| **Integer and math** | The price drifted from dividing before multiplying | AI-1 (rounding over-allocation), AI-3 (triage arithmetic) | Multiply before divide; pro-rata claims. Dust is under 1 token-wei per bidder, and always favours the contract | Figure 6.1; M1 and M7 caught; R1 |
| **Unhandled gas limits** | — | AI-28: a sell-out bid ran out of gas on the wallet's estimate | A fixed bid limit of 200,000. No loops over bidders (claims are pulled). View pages are capped at 50 rows | Figures 6.2 and 6.7; M13 caught; R5 |
| **Stuck or bricked funds** | Burning to address(0) bricked settlement | AI-12, AI-13 (other branch) | start() pulls the funds first; the seller is paid independently of claims; burning uses ERC20Burnable.burn | P2, P3, Figure 6.1 |
| **Inaccurate price calculation** | Drift, and the wrong clearing rule | AI-10 (other branch), AI-20, AI-25, AI-27 | The brief's committed-capital rule, bounds, a frozen display schedule, and honest preview wording | P1, Figure 5.4, the AI-27 screenshots |
| **Front-end provider state** | — | AI-7, AI-14, AI-17 | A real connection check in the end-to-end test; deployment addresses written by tasks and read by the app; refresh before reporting success | Figure 6.5 |

### 5.4 Our final independent review (1 November 2026)

Before submitting, we had Claude (session 06) re-read the final contracts line by line, looking for anything the earlier sessions had not tested, and write five probes

(*evidence/2026-11-01-review/independent-review.probe.js*). We reviewed each finding below. We found no new high-severity defect. The remaining risks are below. Each is inherent to the design or already acknowledged in spec.md, and none puts tracked funds at risk.

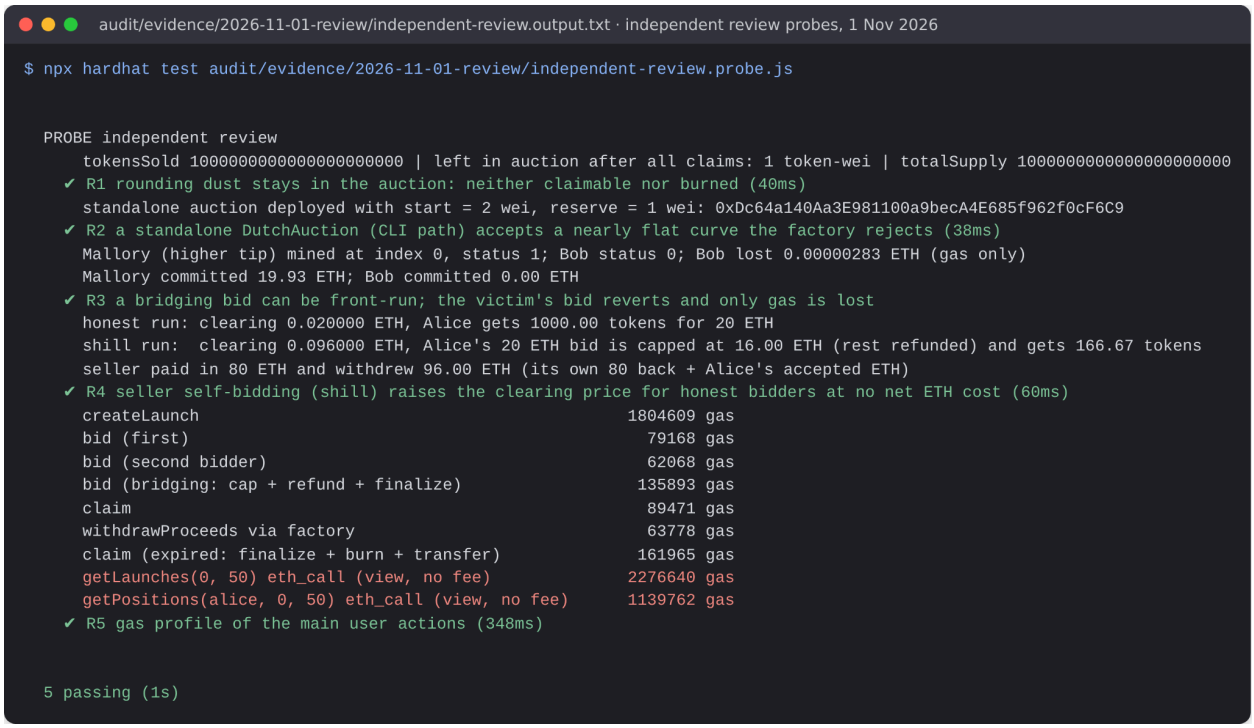



*Figure 5.5 — Probes R1 to R5 on the final build, each reproducing the behaviour described below.*

| # | Finding | Severity | Observed | Our assessment and recommendation |
|---|---|---|---|---|
| R1 | Rounding dust stays in the auction. It is neither claimable nor burned | Informational | 1 token-wei was left after all claims, and totalSupply still counts it | This matches spec section 4 ("below 1 token-wei per bidder, favours the contract"). A future version could add a public sweepDust() that burns the leftovers after every claim |
| R2 | The standalone DutchAuction used by the command-line tasks has none of the factory's bounds | Low | It deployed with a start price of 2 wei and a reserve of 1 wei. The factory rejects both | The website only uses the factory, so users are protected. The command-line path is a developer tool. A future version could move the bounds into the auction's constructor |
| R3 | A bridging bid can be front-run | Informational | A bid with a higher tip was mined first and took the remaining room. The victim's bid reverted, | No funds are lost, because the victim's ETH never left their wallet. This is inherent to any auction in a public mempool. The |

| | | | | |
|---|---|---|---|---|
| | | | costing 0.0000028 ETH in gas | site already warns that "the final split is fixed when your transaction is mined" |
| R4 | A seller can bid on their own listing (shill bidding) | Medium (economic) | The seller committed 80 ETH of its own. The honest bidder's clearing price rose from 0.02 to 0.096 ETH, and the seller withdrew 96 ETH, getting its own 80 ETH back | Listed in spec section 6 under "Known limits": no on-chain rule can stop a seller bidding from another address. We recommend labelling bids from the seller's address in the Activity feed, and showing the seller's share of commitments on the listing page |
| R5 | Gas profile | Informational | createLaunch 1.80 million; first bid 79k; bridging bid (cap, refund and finalize) 136k; claim 89k; claim after expiry, with the burn, 162k; factory withdrawal 64k | Everything is well under block limits, and matches the gas report (Figure 6.7). createLaunch is the expensive step. EIP-1167 clones would cut it about 3.4 times (Section 4.4), at the cost of re-auditing the auction |

Other observations from the review:

- contracts/LaunchTestHelpers.sol uses selfdestruct, which triggers a compiler deprecation warning in the test log. It is test-only: it forces ETH into contracts to prove that forced ETH is never paid out. It is never deployed, and Slither excludes it.
- The curator address is immutable. This removes any path to an admin takeover, but a compromised curator key could still mark a misleading listing "verified". The site shows each listing's seller address next to the badge.
- The frontend's main JavaScript bundle is 972 kB (294 kB compressed), mostly wallet code bundled by RainbowKit. That is fine for a local demo. Loading the wallet modal separately would help a public deployment.

## 6. Validation Process & Testing Strategy

### 6.1 Our strategy: layered, and designed to distrust green results

Reviewing v1 taught us two things: Slither missed 5 of 7 major bugs, and the original tests passed while the contract lost money. So each layer below exists to catch a weakness in the layer above it.

| Layer | What it catches | Tool | Result (final re-run) |
|---|---|---|---|
| 1. Unit, event and revert tests | Wrong behaviour on known paths. They assert ETH **and** token balances | Hardhat, Mocha and Chai | 87 passing |
| 2. Seeded random invariant runs | Paths nobody thought of: the spec section 4 invariants on 10 random auctions, plus random concurrent launches | Mocha loops with fixed seeds | Included in the 87 |
| 3. Reentrancy attackers | The brief's bonus: re-entry from refund, claim and seller callbacks | ReentrancyAttacker.sol, ReentrantSeller | One attempt each, no successes |
| 4. Coverage | Code no test runs | solidity-coverage | 100% of lines; 98.04% of branches |
| 5. **Mutation testing** | Tests that cannot fail | Manual: plant one bug, re-run | 23 of 23 caught (3 after fixes) |
| 6. Static analysis | Known vulnerability patterns | Slither 0.11.6, with a CI gate on high findings | 30 results, 0 high, all triaged |
| 7. Gas report | Costs that grow, and limits a transaction could hit | hardhat-gas-reporter, plus the "Gas limit for bids" tests | Highest bid 152,993 gas, within the 200,000 limit |
| 8. Differential probes | Design errors that both tests and Slither accept | The same scenario on two implementations | P1 to P3 |
| 9. Frontend unit tests | The site's maths drifting from the contract's | Vitest | 39 passing |
| 10. End-to-end UI test | Wallet, site and contracts working together, including layout at 390 px and the bid gas limit | Puppeteer with an injected test wallet | 17 of 17 steps |
| 11. Fresh-clone audit | Anything a marker would hit by following the README | Session 05 | Found AI-26 to AI-28 |
| 12. Our own review in the browser | What no test can judge: appearance, and contradictions on screen | Us, with screenshots | Caught AI-23 and AI-25, and surfaced AI-17 |

| | | | |
|---|---|---|---|
| 13. CI | Regressions on every push | GitHub Actions | 4 jobs |

## 6.2 Contract tests

audit/evidence/2026-11-01-review/test-run.txt · re-run 1 Nov 2026 on the final ZIP

```
$ npx hardhat test
  DutchAuction
    Deployment and start
      ✔ rejects invalid constructor parameters (491ms)
      ✔ does not accept bids before start, and quotes the start price
      ✔ start() pulls the full sale supply and opens a 20-minute window
      ✔ start() is seller-only, single-use and requires token approval
    Price decay
      ✔ decays linearly with no truncation drift (40ms)
      ✔ never increases and never drops below the reserve price
    Bidding and clearing
      ✔ records committed ETH without fixing a per-bid price
      ✔ rejects zero bids and bids after the window closes
      ✔ clears the moment committed capital meets total supply (brief: uniform clearing price)
      ✔ caps a bid that bridges the threshold and refunds its surplus in the same transaction
      ✔ charges every winning bidder the same clearing price
      ✔ sells out on committed capital where per-bid allocation would settle at the reserve
    Expiry and burning
      ✔ settles at the reserve price and burns unsold tokens from totalSupply
      ✔ burns the whole supply when nobody bids
    Claims
      ✔ finalizes automatically on the first claim after expiry
      ✔ rejects early, repeated and empty claims
      ✔ never pays out more than tokensSold when the clearing price rounds down
    Seller proceeds
      ✔ pays the seller exactly the committed ETH, once, without touching bidders' tokens
    Security: reentrancy (bonus)
      ✔ blocks every re-entry from the refund callback and pays only the fair share (44ms)
      ✔ reverts the whole bid when a bidder rejects its refund, leaving state unchanged
    Invariants (seeded random auctions)
      ✔ holds all spec invariants for seed 1 (60ms)
      ✔ holds all spec invariants for seed 2 (63ms)
      ✔ holds all spec invariants for seed 3 (63ms)
      ✔ holds all spec invariants for seed 4 (53ms)
      ✔ holds all spec invariants for seed 5
      ✔ holds all spec invariants for seed 6 (52ms)
      ✔ holds all spec invariants for seed 7
      ✔ holds all spec invariants for seed 8 (49ms)
      ✔ holds all spec invariants for seed 9 (44ms)
      ✔ holds all spec invariants for seed 10 (78ms)

  HortonsToken
    ✔ has the configured name and symbol, and 18 decimals (41ms)
    ✔ mints the whole initial supply to the deployer, once
    ✔ permanently reduces totalSupply when a holder burns
    ✔ does not let anyone burn tokens they neither hold nor were approved for
```

*Figure 6.1 — The DutchAuction and HortonsToken suites (final re-run).*

audit/evidence/2026-11-01-review/test-run.txt · re-run 1 Nov 2026 on the final ZIP (continued)

```
LaunchFactory
  createLaunch
    ✔ moves the whole supply into a running auction and leaves the factory with nothing (67ms)
    ✔ runs the same contract code as a directly deployed auction and token (43ms)
    ✔ emits LaunchCreated with the seller, token and auction
  Parameter bounds
    ✔ rejects an empty name
    ✔ rejects a name over 32 bytes
    ✔ rejects an empty symbol
    ✔ rejects a symbol over 11 bytes
    ✔ rejects zero supply
    ✔ rejects a supply under 1 token
    ✔ rejects a huge supply
    ✔ rejects a zero reserve
    ✔ rejects a reserve under 1 gwei
    ✔ rejects a start price over the cap
    ✔ rejects a start equal to the reserve
    ✔ rejects a nearly flat price curve
    ✔ rejects a raise under 0.001 ETH
    ✔ accepts the extremes of the allowed range, with the price maths intact (52ms)
  Duration and curve
    ✔ gives every launch exactly 1200 seconds, and sellers have no duration or curve input (45ms)
    ✔ cannot be restarted or changed by the seller after creation
  Isolation of concurrent auctions
    ✔ a full lifecycle on one launch changes nothing on another, and the reverse (101ms)
    ✔ burns and totalSupply move independently per launch (41ms)
    ✔ pays one seller's two launches exactly their own amounts (39ms)
  Withdrawing proceeds
    ✔ pays only the registered seller, once, and emits ProceedsForwarded (49ms)
    ✔ cannot withdraw before the auction settles
    ✔ rejects plain ETH from anyone but its own auctions
    ✔ never pays out ETH forced into the factory or an auction (46ms)
  Reentrancy
    ✔ the refund attacker gets no successful re-entry on a factory launch
    ✔ a reentrant seller collects once and cannot reach a neighbour's ETH
    ✔ a seller cannot re-enter the factory to withdraw a second launch mid-withdrawal
    ✔ a seller that rejects ETH blocks only its own withdrawal
  Gas limit for bids
    ✔ a bid estimated as a plain bid runs out of gas on the sell-out path, even at twice the estimate
    ✔ the site's fixed limit covers that bid, and the surplus still comes back
  Listing views
    ✔ pages through 51 listings and clamps the page size to 50 (904ms)
    ✔ reports a row's live state and an account's positions across listings (56ms)
  Curator
    ✔ is the only one who can hide or verify, and the role cannot be reassigned (45ms)
    ✔ rejects a zero curator at deployment
    ✔ curator transactions make no external call and write only to the factory (execution trace) (61ms)
    ✔ cannot move funds or stop an auction by direct calls
    ✔ hiding is not a freeze: a hidden auction runs its whole lifecycle
  Seeded random launches inside the bounds
    ✔ keeps spec section 4 across concurrent auctions (4428ms)

[… 85 lines omitted: Hardhat launchpad and auction task tests, all passing …]

87 passing (11s)
```

*Figure 6.2 — The LaunchFactory suite: bounds, isolation of concurrent auctions, withdrawals, reentrancy, curator limits, the bid gas limit, and random launches. 87 passing in total.*

Some notable tests, and what they prove:

- *"runs the same contract code as a directly deployed auction and token"*: the factory deploys the audited bytecode unchanged, so the auction's earlier evidence still applies.
- *"curator transactions make no external call and write only to the factory (execution trace)"*: the curator provably cannot touch funds.
- *"hiding is not a freeze: a hidden auction runs its whole lifecycle"*: moderation can never trap bidders' ETH.
- *"never pays out ETH forced into the factory or an auction"*: ETH forced in with selfdestruct is ignored, as spec section 4 says.
- *"a bid estimated as a plain bid runs out of gas on the sell-out path, even at twice the estimate"* and *"the site's fixed limit covers that bid, and the surplus still comes back"*: the AI-28 regression tests.

### 6.3 Coverage and static analysis

audit/evidence/2026-11-01-review/coverage.txt · re-run 1 Nov 2026 on the final ZIP

```
$ npx hardhat coverage
File                |  % Stmts | % Branch |  % Funcs |  % Lines |Uncovered Lines |
--------------------|----------|----------|----------|----------|----------------|
 contracts/         |      100 |    98.04 |      100 |      100 |                |
  DutchAuction.sol  |      100 |    96.97 |      100 |      100 |                |
  HortonsToken.sol  |      100 |      100 |      100 |      100 |                |
  LaunchFactory.sol |      100 |      100 |      100 |      100 |                |
--------------------|----------|----------|----------|----------|----------------|
All files           |      100 |    98.04 |      100 |      100 |                |
--------------------|----------|----------|----------|----------|----------------|

> Istanbul reports written to ./coverage/ and ./coverage.json
```

*Figure 6.3a — Coverage (final re-run). The two uncovered branch arms in DutchAuction are both in withdrawProceeds(): a modifier arm, and the "Withdraw failed" revert for a seller that rejects ETH. In the launchpad, that failure is tested one level up, by the factory test "a seller that rejects ETH blocks only its own withdrawal".*

audit/evidence/2026-11-01-review/slither*.txt · re-run 1 Nov 2026 on the final ZIP (Slither 0.11.6)

```
$ slither . --filter-paths "node_modules|ReentrancyAttacker|LaunchTestHelpers" --fail-high
INFO:Slither:. analyzed (20 contracts with 102 detectors), 30 result(s) found

$ slither . --print human-summary
Number of optimization issues: 0
Number of informational issues: 3
Number of low issues: 23
Number of medium issues: 4
Number of high issues: 0

$ echo $?
0      # --fail-high gate passes: no high-severity finding
```

*Figure 6.3b — Slither: no high-severity results, so the CI gate passes.*

In session 05 we found that npm run slither reported 44 results where CI reported 30. The npm script did not skip the two deliberately reentrant test contracts that CI skips. The script now uses the same filter as CI, so a marker running it sees the same 30 results.

**Our triage of the 30 Slither results**

| Detector | Where | Verdict |
|---|---|---|
| incorrect-equality (medium) | accepted == room, startTime == 0, totalCommitted == 0 | A false positive. These compare tracked state, never address(this).balance |
| unused-return (medium) | token.approve in createLaunch | Accepted. The token is the factory's own OpenZeppelin ERC20, which returns true or reverts |
| calls-loop (low) | getLaunches, getPositions | Accepted. These are view functions only, calling contracts the factory deployed, in pages capped at 50 |
| reentrancy-benign (low) | DutchAuction.claim(), which burns before writing claimed when it finalizes | Accepted. The call is burn on our own token, and claim is nonReentrant |
| timestamp (low) | Price and window checks | Accepted. The brief requires a time-bound auction, and each second of validator skew moves the price by about 0.0000667 ETH per token |
| low-level-calls (informational) | The ETH refund, the seller's withdrawal and the factory's forward | Needed to send ETH. Each result is checked, and each function is nonReentrant |

### 6.4 Mutation testing

Mutation testing was the most valuable technique in this project. **It caught three gaps in AI-written tests that 100% line coverage did not**: M7 (AI-1), M2 (AI-21) and M13 (AI-22).

| Contract | Planted bugs | Caught the first time | Caught after closing the gap |
|---|---|---|---|
| DutchAuction (evidence/2026-10-22-findings.md) | 9: price drift, fake burn, double withdrawal, double claim, no guard, double refund, rounding, uncapped bridging bid, wrong clearing price | 8 | 9 |
| LaunchFactory (evidence/2026-10-30-launchpad-mutations.txt) | 14: seller check, guard, balance sweep, curator checks, each bound, receive guard, seller | 12 | 14 |

| | | | |
|---|---|---|---|
| | registration, start omitted, event, page cap, positions | | |

audit/evidence/2026-10-30-launchpad-mutations.txt · manual mutation testing of LaunchFactory.sol

```
M1 no seller check on withdraw: 2 failing
M2 no nonReentrant on withdraw: 0 failing
M3 pays caller balance not tracked amount: 1 failing
M4 no curator check on setHidden: 1 failing
M5 no curator check on setVerified: 1 failing
M6 no reserve floor: 2 failing
M7 no ratio bound: 2 failing
M8 no minimum raise: 1 failing
M9 receive accepts anyone: 1 failing
M10 registers factory as seller: 11 failing
M11 start omitted: 18 failing
M12 no event on forward: 1 failing
M13 page size not clamped: 0 failing
M14 positions report wrong account: 1 failing
M2 no nonReentrant on withdraw (after adding a test): 1 failing
M13 page size not clamped (after adding a test): 1 failing
```

*Figure 6.6 — Factory mutation results. M2 and M13 survived, with 0 tests failing, until we added targeted tests.*

## 6.5 Frontend, end-to-end and gas

audit/evidence/2026-11-01-review/frontend-unit.txt · re-run 1 Nov 2026 on the final ZIP

```
$ npm --prefix frontend test -- --reporter=verbose

 RUN  v5.0.5 /home/claude/final/src/sc4053_group_project_syed_yj-launchpad/frontend

 ✓ src/lib/launch.test.ts > validateLaunch > returns the on-chain values for a good form
 ✓ src/lib/launch.test.ts > validateLaunch > rejects an empty name
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a name over 32 bytes
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a long name in multi-byte characters
 ✓ src/lib/launch.test.ts > validateLaunch > rejects an empty symbol
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a symbol over 11 bytes
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a supply that is not a number
 ✓ src/lib/launch.test.ts > validateLaunch > rejects zero supply
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a supply under 1 token
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a huge supply
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a zero reserve
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a reserve under 1 gwei
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a start price over the cap
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a start equal to the reserve
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a nearly flat curve
 ✓ src/lib/launch.test.ts > validateLaunch > rejects a raise under 0.001 ETH
 ✓ src/lib/launch.test.ts > validateLaunch > accepts the smallest allowed raise
 ✓ src/lib/launch.test.ts > parseRoute > maps hashes to pages
 ✓ src/lib/launch.test.ts > parseRoute > treats anything else as not found
 ✓ src/lib/auction.test.ts > priceAt > matches the contract's checkpoints with no truncation drift
 ✓ src/lib/auction.test.ts > priceAt > quotes the start price before the auction starts
 ✓ src/lib/auction.test.ts > previewBid > caps a bid that bridges the threshold and refunds the surplus
 ✓ src/lib/auction.test.ts > previewBid > accepts a bid in full while there is room
 ✓ src/lib/auction.test.ts > sell-out timing > 48 ETH sells out 1,000 TIM when the price reaches 0.048 ETH, at t = 780 s
 ✓ src/lib/auction.test.ts > sell-out timing > returns null when the committed ETH cannot sell out above the reserve
 ✓ src/lib/auction.test.ts > sell-out timing > marks the auction sold out between blocks, without waiting for a transaction
 ✓ src/lib/auction.test.ts > sell-out timing > reports an expired, undersubscribed auction as ended
 ✓ src/lib/auction.test.ts > token estimates > gives a guaranteed minimum at the current price, and the pro-rata share after sell-out
 ✓ src/lib/auction.test.ts > token estimates > maps a bid price back to its time for the chart
 ✓ src/lib/simulate.test.ts > scheduleElapsed > follows the clock while an auction is running
 ✓ src/lib/simulate.test.ts > scheduleElapsed > stops at the sell-out, however much later it is read
 ✓ src/lib/simulate.test.ts > scheduleElapsed > stops at 20 minutes if the auction never sells out
 ✓ src/lib/simulate.test.ts > project > predicts a sell-out and a pro-rata share when the committed ETH is enough
 ✓ src/lib/simulate.test.ts > project > predicts settling at the reserve when it is not
 ✓ src/lib/simulate.test.ts > project > gives nothing to a bidder with no stake
 ✓ src/lib/simulate.test.ts > simulate > keeps running before the price meets the committed ETH
 ✓ src/lib/simulate.test.ts > simulate > sells out where the price meets the committed ETH, at one price for everyone
 ✓ src/lib/simulate.test.ts > simulate > settles at the reserve and burns the rest if time runs out
 ✓ src/lib/simulate.test.ts > simulate > refunds what no longer fits

 Test Files  3 passed (3)
      Tests  39 passed (39)
   Start at  12:09:17
   Duration  1.22s (import 73%, transform 21%, tests 4%, worker 2%)
```

*Figure 6.4 — The 39 frontend unit tests: form validation mirrors the contract's bounds, the site's maths matches the contract's checkpoints "with no truncation drift", and scheduleElapsed freezes at the sell-out (AI-25).*

audit/evidence/2026-11-01-review/e2e.txt · re-run 1 Nov 2026 on the final ZIP (headless Chromium, injected test wallet)

```
$ npm --prefix frontend run e2e
  ✓ seller connected with no sign-up
  ✓ form explains invalid values before the wallet opens
  ✓ seller launched Alpha Coin in one transaction and landed on its page
  ✓ bidder sees the listing on the market
  ✓ 20 ETH bid confirmed, with a gas limit that covers the sell-out path
  ✓ 100 ETH bid capped, surplus refunded, auction settled
  ✓ bidder claimed tokens, and the wallet was asked to add them automatically
  ✓ seller withdrew the proceeds
  ✓ result card shows 1,000 ALP sold, under the listing's own symbol
  ✓ second listing expired, settled at the reserve, burned 500 BET, and left the first listing alone
  ✓ bidder's portfolio lists bids across both listings, with claim available in place
  ✓ portfolio tracks a starred coin live, and a bid went through from there
  ✓ curator hid and verified a listing
  ✓ hidden listing left the market, and its bidder could still claim
  ✓ no horizontal scroll on any page at 390px
  ✓ the How it works simulator plays an auction through to its result
  ✓ theme toggle works and is remembered
Passed. Screenshots in /home/claude/final/rerun/e2e-screenshots
```

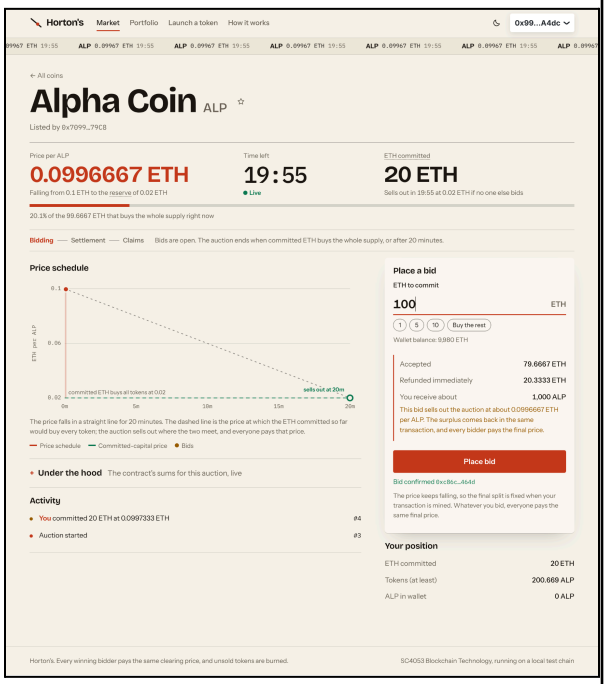
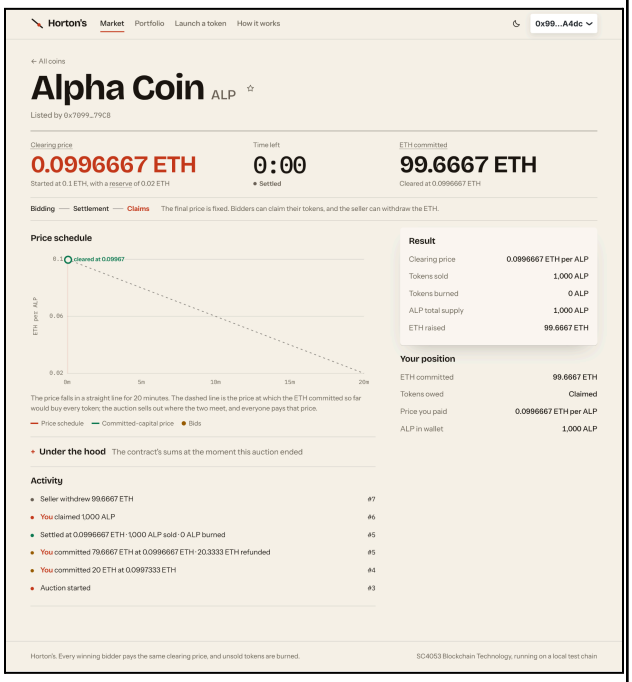
*Figure 6.5 — The end-to-end test through the real site: 17 steps, three listings, and the seller, bidder and curator wallets (final re-run, headless Chromium).*

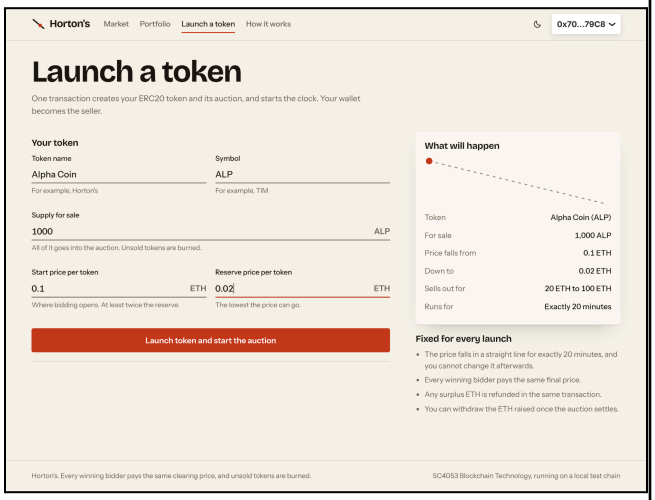
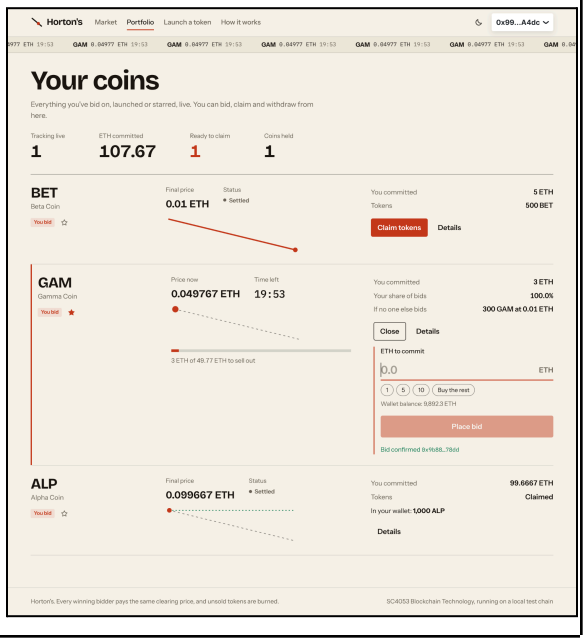
audit/evidence/2026-10-31-gas-report.txt · hardhat-gas-reporter, all 87 tests

| Solc version: 0.8.24 · Optimizer enabled: true · Runs: 200 · Block limit: 60000000 gas | | | | | | |
|---|---|---|---|---|---|---|
| Methods | | | | | | |
| Contract | Method | Min | Max | Avg | # calls | usd (avg) |
| DutchAuction | bid | 44968 | 152993 | 88776 | 458 | - |
| DutchAuction | claim | 84671 | 161965 | 89787 | 219 | - |
| DutchAuction | finalize | 82480 | 111980 | 105005 | 39 | - |
| DutchAuction | start | - | - | 96031 | 8 | - |
| DutchAuction | withdrawProceeds | 32930 | 39630 | 39134 | 27 | - |
| LaunchFactory | createLaunch | 1787449 | 1804681 | 1789940 | 314 | - |
| LaunchFactory | setHidden | 30443 | 47555 | 42666 | 7 | - |
| LaunchFactory | setVerified | 25716 | 30528 | 28924 | 6 | - |
| LaunchFactory | withdrawProceeds | 63778 | 63790 | 63789 | 88 | - |

*Figure 6.7 — Gas report over all 87 tests (evidence/2026-10-31-gas-report.txt). The most expensive bid, a sell-out, costs 152,993 gas, inside the 200,000 limit the site sends. No cost grows with the number of bidders.*

Screenshots from the end-to-end run show the core mechanics on screen:

| **The bridging-bid preview, before signing** | **After settlement: one clearing price, the claim and the withdrawal** |
| --- | --- |
|  |  |
| A 100 ETH bid: most is accepted, and the surplus is refunded in the same transaction. This is the brief's tricky point about a bid that bridges the clearing threshold | The clearing price is fixed and 1,000 ALP are sold. The activity feed shows the capped bid, the refund, the settlement, the claim and the seller's withdrawal |

| The Launch form, which checks the bounds before the wallet opens | The Portfolio, tracking every listing live |
|---|---|
|  |  |



## 6.6 Reentrancy resistance (the brief's bonus), before and after

| | Our v1 (hand-written) | Final |
|---|---|---|
| An attacker re-enters during the ETH callback | It succeeds once, and leaves with **8 ETH + 200 NTU** instead of the fair 4 ETH + 100 NTU | Every re-entry reverts, and the attacker gets exactly its fair share |
| Why | hasClaimed was set after the transfer and the ETH call | State is written before every external call, and bid, claim and withdrawProceeds are nonReentrant |
| Re-entry by a seller | — | ReentrantSeller tries to re-enter the factory for a second launch in the middle of a withdrawal, and is blocked (AI-21) |
| Evidence | evidence/2026-10-22-findings.md | The test *"blocks every re-entry from the refund callback…"* (Figure 6.1), three factory reentrancy tests (Figure 6.2), and mutations M5 (auction) and M2 (factory) caught |

## 6.7 Limits of our evidence

We state these plainly so the evidence can be weighed fairly:

- **No public testnet deployment.** We chose to deploy locally, which the submission instructions allow, so there are no Sepolia transaction hashes. YJ's

earlier Sepolia deployments on dutch-auction-token-launch are of the per-bid contract, and are not evidence for this one.

- **The test wallet.** The automated end-to-end test uses an injected test wallet, not the MetaMask extension. We used real MetaMask on the single-auction version in sessions 02 to 04: bids, a bridging bid of which 81.03 ETH was accepted at 0.0865 ETH, and claims. We use MetaMask for the launchpad in our demo video. AI-28 showed why this matters: the test wallet's unlimited gas hid a whole class of bug.
- **Manual mutation testing.** We planted the bugs by hand rather than with a framework such as Gambit, so the set of mutations reflects our judgement.
- **No formal verification or Foundry invariant fuzzing.** Our random runs use fixed seeds. They are reproducible, but bounded.
- **The MetaMask diagnosis (AI-19)** rests on reading the extension's code and on the error stopping, not on a reproduction inside the extension.

## 7. Improvements & Refinements

### 7.1 Changes beyond the base AI output, and who originated each

| Change | Type | Originated by | Evidence |
|---|---|---|---|
| Strict adherence to the brief's clearing rule, rejecting our v1 model and the other branch's | Architecture | **Us** | S01 P2, S03 |
| Pro-rata claims, replacing the AI's per-bid formula | Correctness | The AI, after mutation testing | AI-1 |
| Two spec invariants corrected (forced ETH, dust bids) | Specification | The AI, after review | Spec section 4 |
| Fixed-height stat cards, because live numbers made the layout jump | UX | **Us** (S02 P4) | styles.css |
| Comparing and combining the two branches, keeping P1 as a regression test | Architecture | **Our** request, carried out by the AI | S03 |
| Scope raised from one auction to a **launchpad**, and narrowed from an "exchange" | Features | **Our** request. The AI narrowed it against the rubric | S04 P2 |
| Factory bounds hardened, and the curator's pause power removed | Security | The AI's strict-marker agent | AI-20 |
| Local deployment instead of Sepolia, and a website-only demo | Scope | **Us** | S04 P3, P6 |
| A live portfolio tracker, with bid, claim, settle and withdraw in place | Features | **Us** (S04 P7) | Section 6.5 screenshots |
| The token offered to the wallet automatically after a claim (wallet_watchAsset) | Features | **Us** | End-to-end step 7 |
| A How it works page with an auction simulator, live contract sums under "Under the hood", and a glossary | Features and UX | **Us** | Figure 5.4 |
| A full redesign, with a light theme, a live market chart and a price ticker | UX | **Us** ("looks very basic and AI-generated") | AI-23, Figure 7.1 |
| The display schedule frozen after a sell-out | Correctness | **We** spotted it, and the AI fixed it | AI-25 |
| MetaMask nonce instructions in the README | Documentation | Our problem, the AI's diagnosis | AI-19 |
| A final fresh-clone audit against the brief, before filming | Process | **Us** (S05 P1) | AI-26 to AI-28 |
| A fixed bid gas limit, and honest "about" wording in the sell-out preview | Correctness | The AI, from that audit | AI-27, AI-28 |

Our part in building the code was mainly directing and reviewing rather than typing. We read each change before approving its commit. After each session we ran the site ourselves, in the browser and with MetaMask, and fed back what we saw as screenshots and prompts (sessions 02 to 04). Several of the improvements above came from that loop.

### 7.2 Architectural changes relative to our v1

| Our v1 (hand-written) | Final | Effect |
|---|---|---|
| The clock starts in the constructor | start() pulls the funds, then starts the clock | No unfunded auctions |
| Per-bid allocation, with refunds at claim time | Committed capital, with one same-transaction refund path | Follows the brief, and removes the double-refund class of bug |
| Burn by transfer to address(0) | ERC20Burnable.burn | totalSupply really falls |
| The seller sweeps address(this).balance | The seller is paid exactly totalCommitted, once | No drain, and forced ETH is ignored |
| Several settlement paths | One internal _finalize() | Always emits AuctionFinalized |
| An Ownable token | No owner and no mint function | A smaller trust surface |
| One auction, from the command line only | A factory, any number of isolated auctions, and a website | A launchpad, with tests that prove the auctions are isolated |

### 7.3 Optimisations

1. **A stateless price,** computed from block.timestamp, multiplying before dividing: no storage writes and no drift.
2. **No loops over bidders.** Claims are pulled, so claim() costs about 89k gas however many people bid.
3. **Paged views** replace a backend or an event indexer: 2.28 million gas for 50 rows, as a free eth_call.
4. **Bounded inputs** (MAX_SUPPLY of 1e15 tokens, MAX_PRICE of 1,000 ETH) keep supply × price far below 2^256.
5. **Immutable parameters** in the auction save a storage read on every price check.
6. **A fixed bid gas limit.** Unused gas is refunded, so the limit costs the bidder nothing and removes a failure mode.
7. **Refreshing before reporting success** in useTx removed a race in the site and an intermittent CI failure.

## 7.4 The UI before and after our review

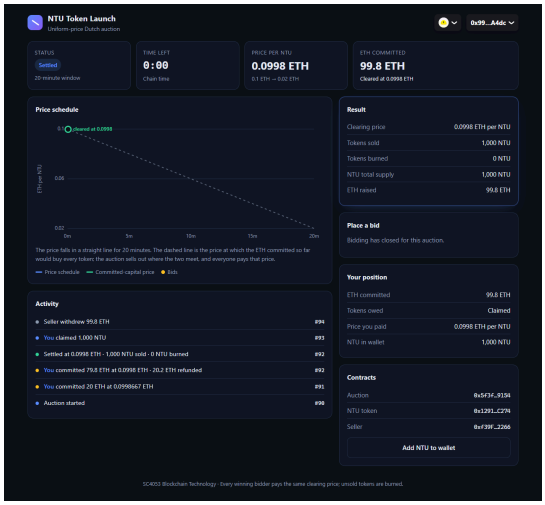
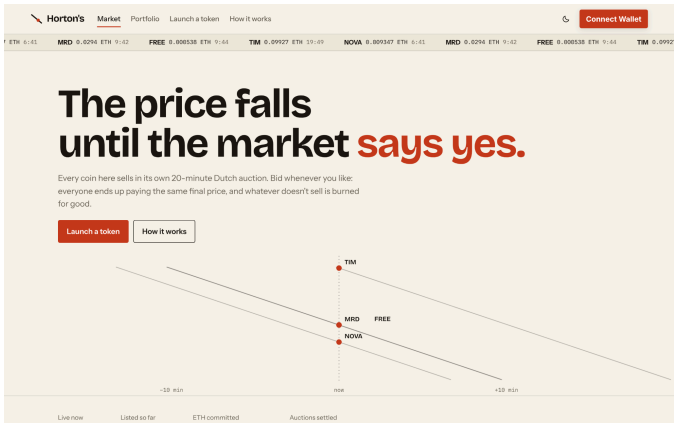
| Before: the AI's first UI (23 October) | After our redesign request (30 October onwards) |
| --- | --- |
|  |  |
| A single auction in boxed dark cards. We called it "very basic and AI-generated" | The launchpad market: a live price chart, a ticker, filters, verified badges and six listings in every state |





*Figure 7.1 — The redesign came from our own judgement. No functional test could have flagged the original.*

## 8. Lessons Learned

### 8.1 Where AI excelled

- **Speed of investigation.** In one session, the AI turned "0 of 4 tests failing" into isolated root causes, probes for three hidden money bugs, a redesign and 29 tests. That would have taken us days by hand.
- **Experiments instead of opinions.** It compared designs by running them: the same bids on two contracts (P1 to P3), and clones against new, measured at 3.4 times. It did not argue in the abstract.
- **Breadth of tooling.** It set up coverage, Slither, mutation testing, a gas report, CI and an end-to-end browser test with an injected wallet. We would probably have skipped most of that infrastructure on our own.
- **Tracing unfamiliar code.** It followed a false MetaMask error into the extension's minified code, and found a plausible cause that was nowhere in our dApp.
- **Adversarial agents.** Asking one agent to refute another overturned a confident wrong claim (AI-19), and caught weak bounds before they reached our repository (AI-20).
- **Auditing its own work when asked.** Its fresh-clone audit in session 05 found three of its own mistakes that 120 passing tests had missed (AI-26 to AI-28).

### 8.2 Where AI failed

- **It states guesses as facts.** Authorship (AI-2, AI-16), commit ancestry (AI-15), package versions (AI-9), and a claim about MetaMask's internals from a truncated read (AI-19). Sounding fluent is not evidence.
- **It writes tests that agree with its code.** Round-number tests hid AI-1. A swallowed assertion and blind tests hid AI-22. A text check that also matched a disconnected page hid AI-7. Its "all green" could not be trusted until mutation testing challenged it.
- **It satisfies the compiler, not the requirement.** It chose bounds only to avoid reverts (AI-20), and dropped a guard because one attack path was blocked (AI-21).
- **It checks the happy state.** It tested the price display only on live auctions (AI-25), the layout only at desktop width (AI-8, AI-24), the seed task with only one value (AI-26), and gas only with a wallet that never runs out (AI-28).
- **It repeats its own reasoning errors in new code.** AI-27 is AI-6 again, three sessions later, and AI-16 is AI-2 again.
- **It cannot judge taste.** It called a generic UI "clean" (AI-23). We had to tell it otherwise.
- **Unsupervised, the other tool did worse.** The AI-generated branch, built with no spec and no audit loop, had five defects, three of them high. They included a

design that underpaid the seller by 72% on identical bids, and a build that failed from a fresh clone.

### 8.3 What made the difference

1. **A spec that quotes the brief, clause by clause.** Both our v1 and the other branch got the clearing rule wrong. The table mapping the brief to rules made that mistake impossible to miss.
2. **Coding rules aimed at known failure modes** (spec section 5). They caught AI-21 and AI-22, and gained a gas rule after AI-28.
3. **Mutation testing.** It was the only layer that caught all three gaps in the AI's tests.
4. **Differential testing against a second implementation.**
5. **A running log of AI mistakes, with prevention rules.** The authorship rule worked the second time (AI-16).
6. **Testing in conditions that match real use.** A fresh clone, the real seed command and a realistic gas limit exposed AI-26 to AI-28. The test environment had hidden all three.
7. **A human in the loop.** Four of the sixteen medium and high issues from our own sessions were first noticed by one of us, two of them (AI-23 and AI-25) by looking at the running app.

### 8.4 Our recommendations for future AI-driven dApp development

1. Write the spec **before** prompting, and make it quote the requirements word for word.
2. Never accept "tests pass" from the tool that wrote the code. Run mutation testing, or have a separate agent attack it.
3. Keep provenance and intent out of the AI's hands, and record them yourself.
4. Ask for differential evidence: run the same scenario against alternatives, including a deliberately naive one.
5. Make the AI verify facts it could check (npm view, git log --format=%P, reading the actual source), and make it say so when it is guessing.
6. Test the end states, not just the happy path: sold out, expired, hidden, mobile width, dark theme, and the real gas a real wallet would send.
7. Before submitting, have the AI audit its own work from a fresh clone, as a marker would.
8. Use static analysis as one layer, not as the safety net. On our project, Slither missed most of the logic bugs.
9. Keep a human design review in the loop. UX is assessed, and tests cannot see it.

## Appendix A — Evidence index

| File | What it shows |
|---|---|
| audit/evidence/2026-10-22-baseline-test-run.txt | Our hand-written v1: 0 passing, 4 failing |
| audit/evidence/2026-10-22-baseline-slither.txt | v1: 19 Slither results, including reentrancy-eth |
| audit/evidence/2026-10-22-findings.md | Probes for v1's hidden bugs; the v2 mutation table (M1 to M9); the v2 Slither triage |
| audit/evidence/2026-10-22-v2-*.txt | v2: 29 passing; 100% lines and 96.97% branches; Slither |
| audit/2026-10-22-issues-report.md | The full review of v1 (13 issues), exported from the live document (revision 50) |
| audit/evidence/2026-10-23-ui/ | Screenshots of the first UI, before the redesign |
| audit/evidence/2026-10-28-branch-comparison.md | Side-by-side comparison of the two implementations |
| audit/evidence/2026-10-28-probes/ | Probe scripts and output for P1 to P3 |
| audit/evidence/2026-10-28-combined-*.txt | The combined branch: 39 passing, and coverage |
| audit/evidence/2026-10-30-launchpad-* | Launchpad tests, coverage, Slither and its triage, mutations M1 to M14, and the end-to-end run |
| audit/evidence/2026-10-30-ui/ | Launchpad screenshots in the light and dark themes |
| audit/evidence/2026-10-31-*.txt | Session 05: 87 tests, coverage, Slither, the end-to-end run and the gas report |
| audit/evidence/2026-10-31-ui/ | AI-27 before and after screenshots |
| audit/evidence/2026-11-01-review/ | Our final re-run on the submitted code: tests, coverage, Slither, frontend tests, the end-to-end run with screenshots, probes R1 to R5, and demo screenshots |
| audit/figures/ | The diagrams and log figures used in this report |
| audit/ai-issues.md | All 28 AI mistakes in the four-part format |
| prompts/2026-1*-session-0*.md | Prompt logs for sessions 01 to 06 |

**Appendix B — Reproducing the evidence**

```
npm ci && npm --prefix frontend ci
 npm test                          # 87 passing
 npm run coverage                  # 100% lines, 98.04% branches
 npm run slither                   # 30 results, 0 high (same filter as CI)
 npm --prefix frontend test        # 39 passing
 npm --prefix frontend run build
  npx hardhat test audit/evidence/2026-11-01-review/independent-review.probe.js     # probes R1 to R5


 # End-to-end: terminal 1: npm run node
 # terminal 2: npx hardhat launchpad:deploy --network localhost && npm run frontend


 npm --prefix frontend run e2e             # 17 steps
```

Our final re-run used Node 22, solc 0.8.24 and Slither 0.11.6 on Linux, with Playwright's Chromium passed to the end-to-end script through BROWSER_PATH.

## Appendix C — Brief and rubric compliance map

### Brief section 2.1: required sections

| Required section | Where |
|---|---|
| 1. Initial Specification & System Scope (requirements before AI, contract logic, UI) | Sections 1.1 to 1.3 |
| 2. Development Methodology & AI Tooling (assistants; custom agents, scripts and spec files; division of labour) | Sections 2.1 to 2.4 |
| 3. Specification Engineering & Prompt Logs (formal spec files; prompt chains, iterations and context windows) | Sections 3.1 to 3.4, and prompts/ |
| 4. AI-Generated Architecture (structure, interfaces, state management) | Sections 4.1 to 4.4 |
| 5. Problems & Vulnerability Analysis (a comprehensive inventory) | Sections 5.1 to 5.4, and ai-issues.md |
| 6. Validation Process & Testing Strategy | Sections 6.1 to 6.7 |
| 7. Improvements & Refinements | Sections 7.1 to 7.4 |
| 8. Lessons Learned | Sections 8.1 to 8.4 |

**Brief section 2.2: the major issue format.** Every major issue in Section 5.2, and every entry in ai-issues.md, states why the AI made the mistake, why it is problematic, how it was fixed, and the evidence that the fix works, with a log, a test name, a mutation result or a screenshot.

### Option 2's tricky points

| Tricky point | Our answer | Evidence |
|---|---|---|
| A block-based countdown and continuous price decay | The price is derived from block.timestamp on every call, multiplying before dividing | "decays linearly with no truncation drift" (Figure 6.1) |
| Claims when a bid bridges the clearing threshold | The bridging bid is capped at room, the auction finalizes in the same transaction, and the surplus is refunded | The Section 6.5 screenshots; "caps a bid that bridges the threshold…" |
| Safe ETH refunds against reentrancy | One refund path, checks-effects-interactions, nonReentrant, and attacker contracts | Section 6.6 |
| Permanently burning unsold tokens | ERC20Burnable.burn on the auction's own balance, with totalSupply asserted | "settles at the reserve price and burns unsold tokens from totalSupply"; the end-to-end step "burned 500 BET" |

**The rubric's "Development & AI Audit Quality" row.** Its high standard asks for "rigorous evaluation of AI agents/tools, prompt specs, and vulnerability remediation".

We evaluate agents and tools in Sections 2 and 8, prompt specs in Section 3, and vulnerability remediation in Section 5, with reproducible evidence in Section 6 and Appendix B.